ALX Software Engineering Programme

GUIDE AND RESOURCES

S1

By TALIS

This file is not official from ALX

This file contains links and resources that make it easy for you to follow ALX program for software engineering.

Most of the links were compiled by the following source:

https://alx-feb-resources.notion.site/ALX-Resources-1660799bece74b2f9afb1bcc24dff313

It is a team of Egyptian youth. Their account links are below:

Yousef Abodawoud

https://www.linkedin.com/in/abodawoud/

https://twitter.com/Abodaawoud

Emad Anwer

https://www.linkedin.com/in/emadanwer/

https://twitter.com/EmadAnwer_

Kholoud Fattem

https://www.linkedin.com/in/kholoudxs55kh/

https://twitter.com/Kholoudxs55kh

Abdelmaged Seif (Maged)

https://www.linkedin.com/in/abdelmageed-seif/

https://twitter.com/maged_saif1

Shaza Aly

https://www.linkedin.com/in/shazaali/

https://twitter.com/ShazaAlyOthman

Abeer Mosaad

https://www.linkedin.com/in/abeer-mosaad-845a32230/

https://twitter.com/Abeer_MOsaad0

The links in this file are organized by **TAHIRI KHALID (TALIS)**:

https://www.linkedin.com/in/khalid-tahiri

https://twitter.com/talistech

https://github.com/TALIS-PRO

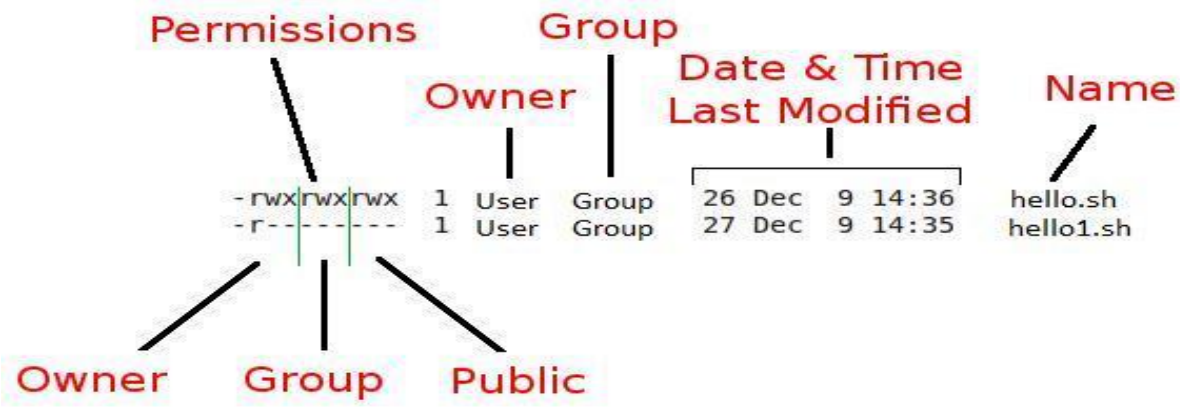
Assembling and organizing all this links consumes a lot of effort and time

Don't forget to follow us 👍

Table of Contents

Month#0

Shell permissions



it’s about the permission the user gives for the files and how to change the permissions and ownership of it.

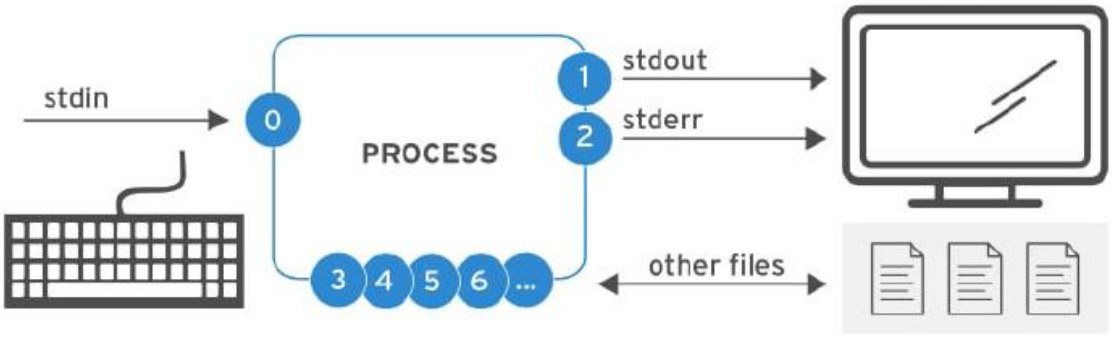
- **Resources:**

https://youtu.be/4e669hSjaX8

https://www.geeksforgeeks.org/permissions-in-linux/

https://youtu.be/07JOqKOBRnU

Shell I/O Redirections and filters



In this Task you get to know about how redirection happens in Command lines using redirection or piping, and more.

- **Resources:**

https://youtu.be/skTiK_6DdqU

https://youtu.be/5EqL6Fc7NNw

https://www.youtube.com/watch?v=oPEnvuj9QrI

https://www.youtube.com/watch?v=Tc_jntovCM0

Shell, init files, variables and expansions

it's about variables, alias and how to ease your command lines.

https://youtu.be/SP0gDpamofY

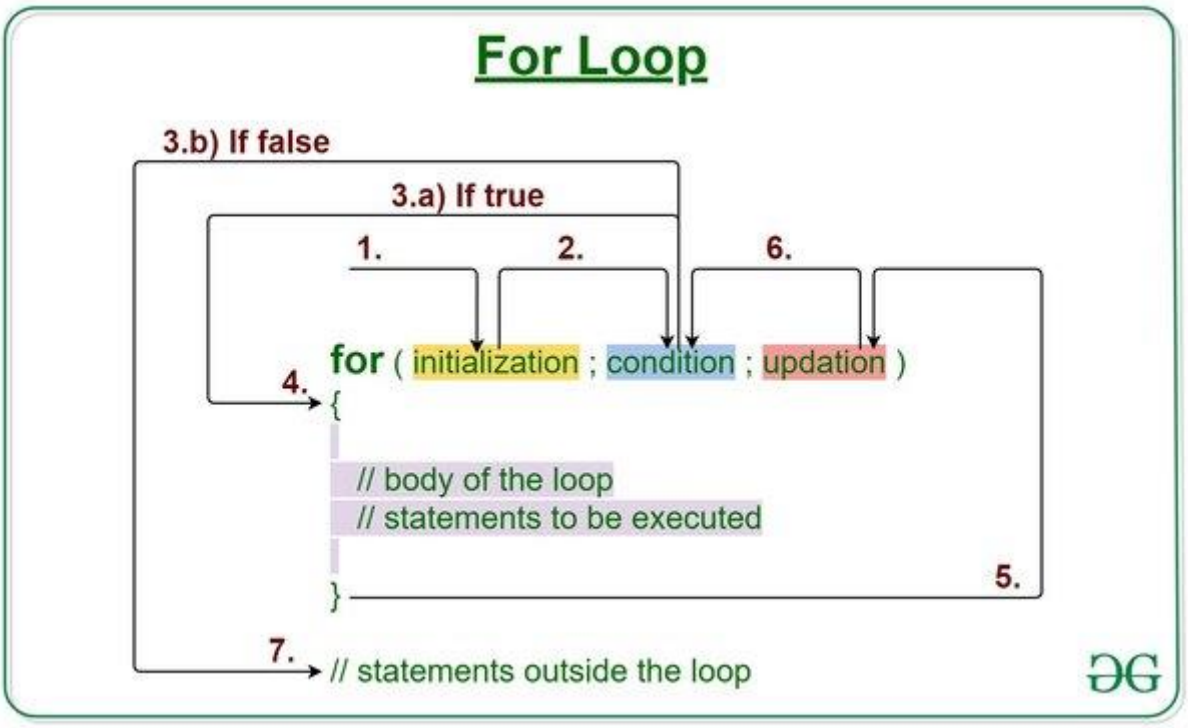
How to Setup the ALX Environment By using WSL & VSCODE

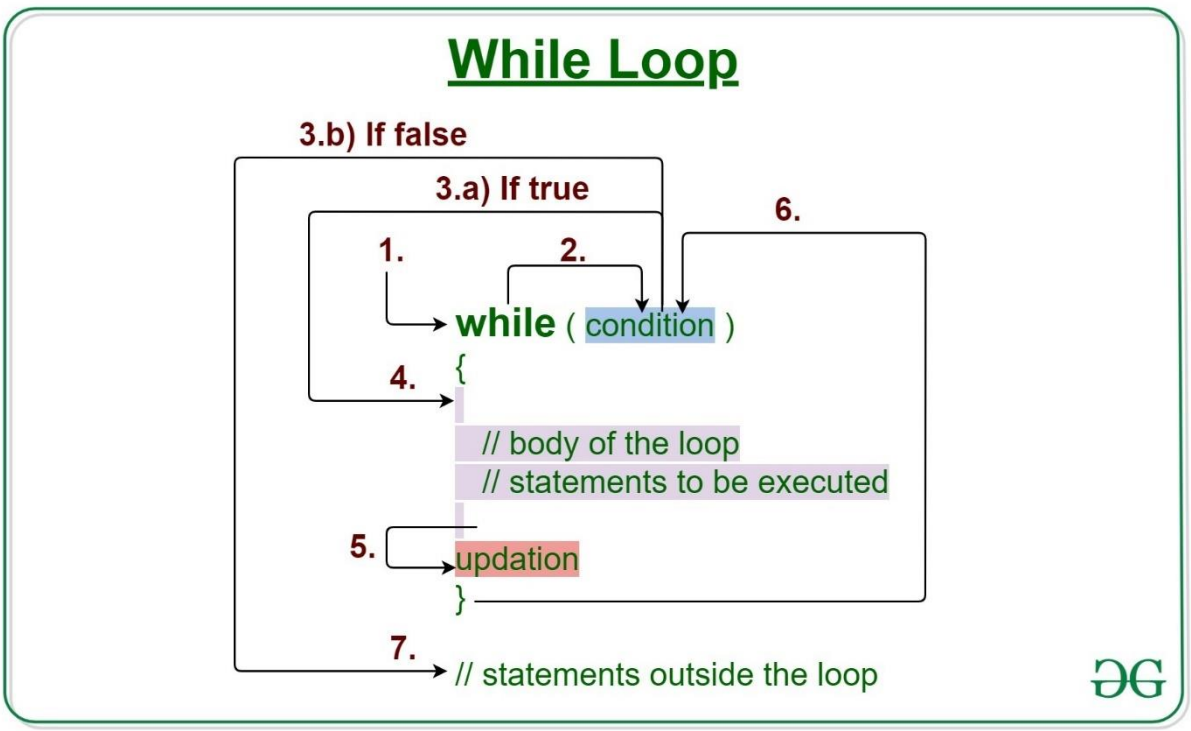
As you are now free to connect your Sandbox with your PC, or to create a suitable environment for your work, here's a suggestion for you to have a good helpful and easy environment.

https://youtu.be/w9Rhi2WR8Js

C Basics, Variables and Loops

They consist of the first tasks in C Language, What's C, How to declare Variables and what are loops in C.





- **Resources For C:**

https://youtube.com/playlist?list=PLBlnK6fEyqRggZZgYpPMUxdY1CYkZtARR

https://youtu.be/KJgsSFOSQv0

https://youtube.com/playlist?list=PLtFbQRDJ11kFbr3rN-53qmMD-k0USosqx

https://youtu.be/87SH2Cn0s9A

- **Resources For Variables:**

https://youtu.be/dhh5lrXXXYw

https://youtu.be/fO4FwJOShdc

https://youtu.be/4UFDAOty4WQ

https://youtu.be/zBhT544Xy0o

- **Resources For Loops:**

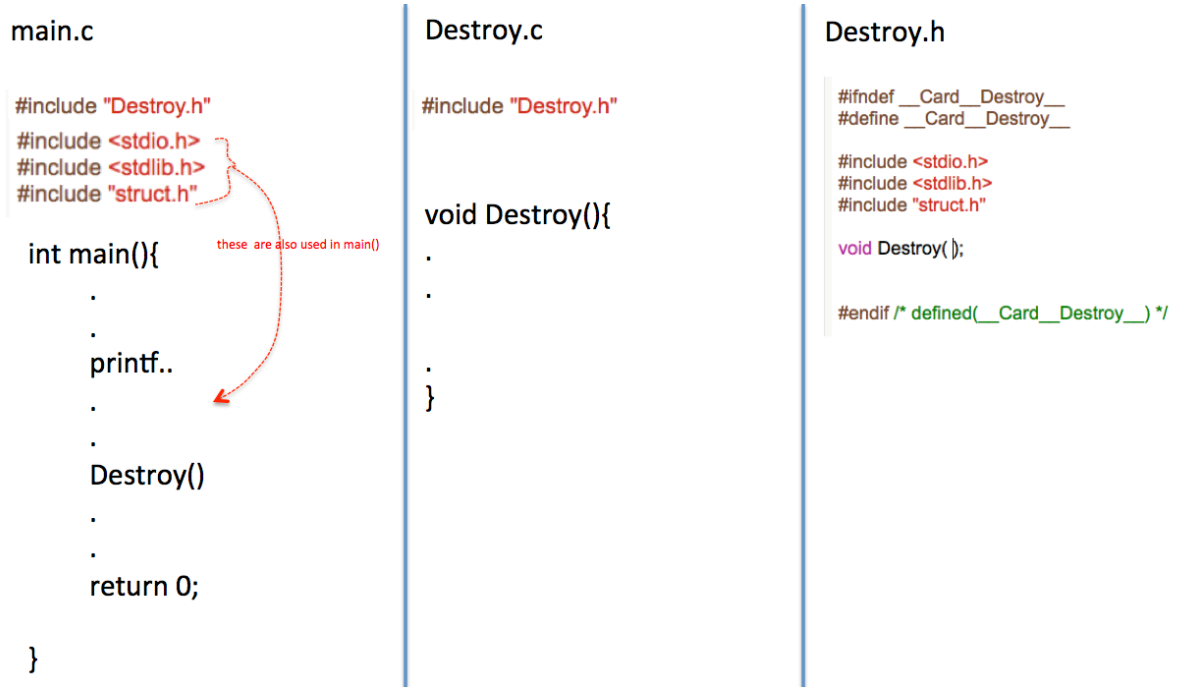
https://youtu.be/cuh-5AE0AHs

https://youtube.com/playlist?list=PLBlnK6fEyqRgZq4a-SMViZr-V8jlvCioJ

https://youtu.be/BpeIfof3VBk

C Functions and Header Files

In These Tasks you get to know about Functions, what they are, how are they created, how the functions in Standard Libraries work, and what are the header files, how to create them and how to make your own header file with your own functions.



- **Resources For Functions:**

https://youtube.com/playlist?list=PLBlnK6fEyqRi0Va6znG73P52rFfXD5fhs

https://youtu.be/OcjBc_1e3M8

https://youtu.be/oMXhxEpznoc

https://youtu.be/fdauhXJOuDE

https://youtu.be/Npo1u37lcg8

https://youtu.be/vc9A6HdrTz4

https://youtu.be/g2VyJYrcKgY

- **Resources For Header Files:**

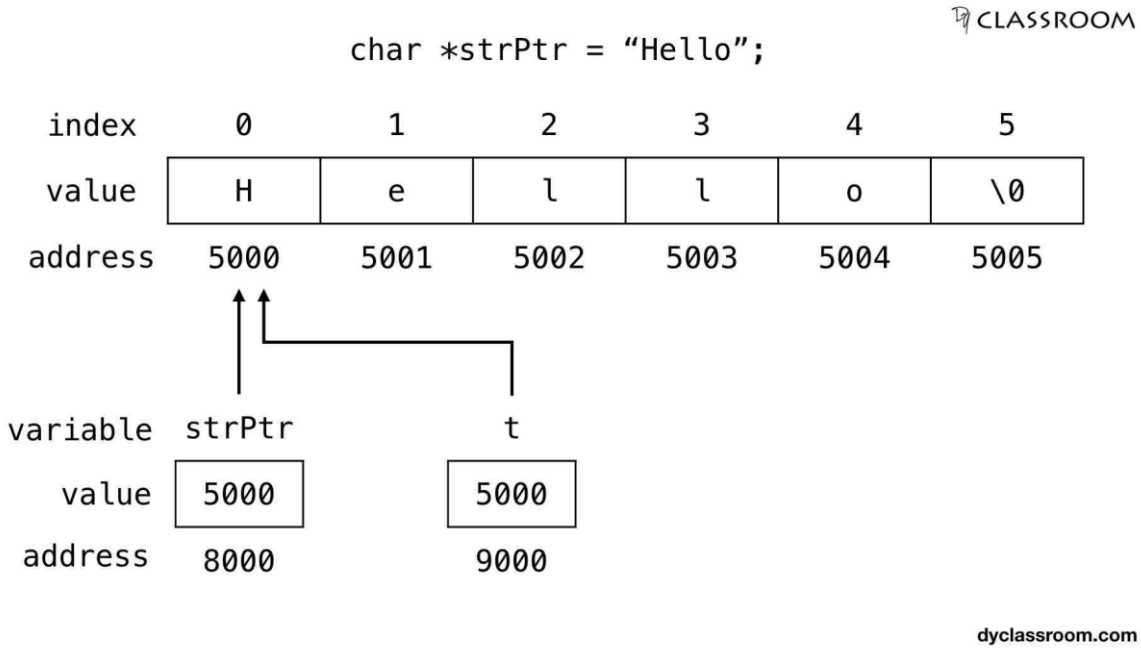
https://youtu.be/u1e0gLoz1SU

https://youtu.be/oe11Dhw9dOg

https://youtu.be/EtZLh-gxYqQ

C - Pointers, arrays and strings

This topic consists of 3 projects related to pointers, here are the resources for each topic.



- **Resources for Pointers:**

https://www.youtube.com/watch?v=lgcFYFbzCL8&list=PLnrlZUDQofUv7JE4QIahAyztrQU9bnJmd&index=39

https://www.youtube.com/watch?v=zuegQmMdy8M

https://youtube.com/playlist?list=PLBlnK6fEyqRjoG6aJ4FvFU1tlXbjLBiOP

https://www.scaler.com/topics/c/memory-layout-in-c/

https://www.youtube.com/watch?v=jUcqT37FdUI

- **Resources for Arrays and strings:**

https://www.youtube.com/watch?v=yhgpqUz3Hos&list=PLnrlZUDQofUv7JE4QIahAyztrQU9bnJmd&index=40

http://youtube.com/watch?v=Qp3WatLL_Hc

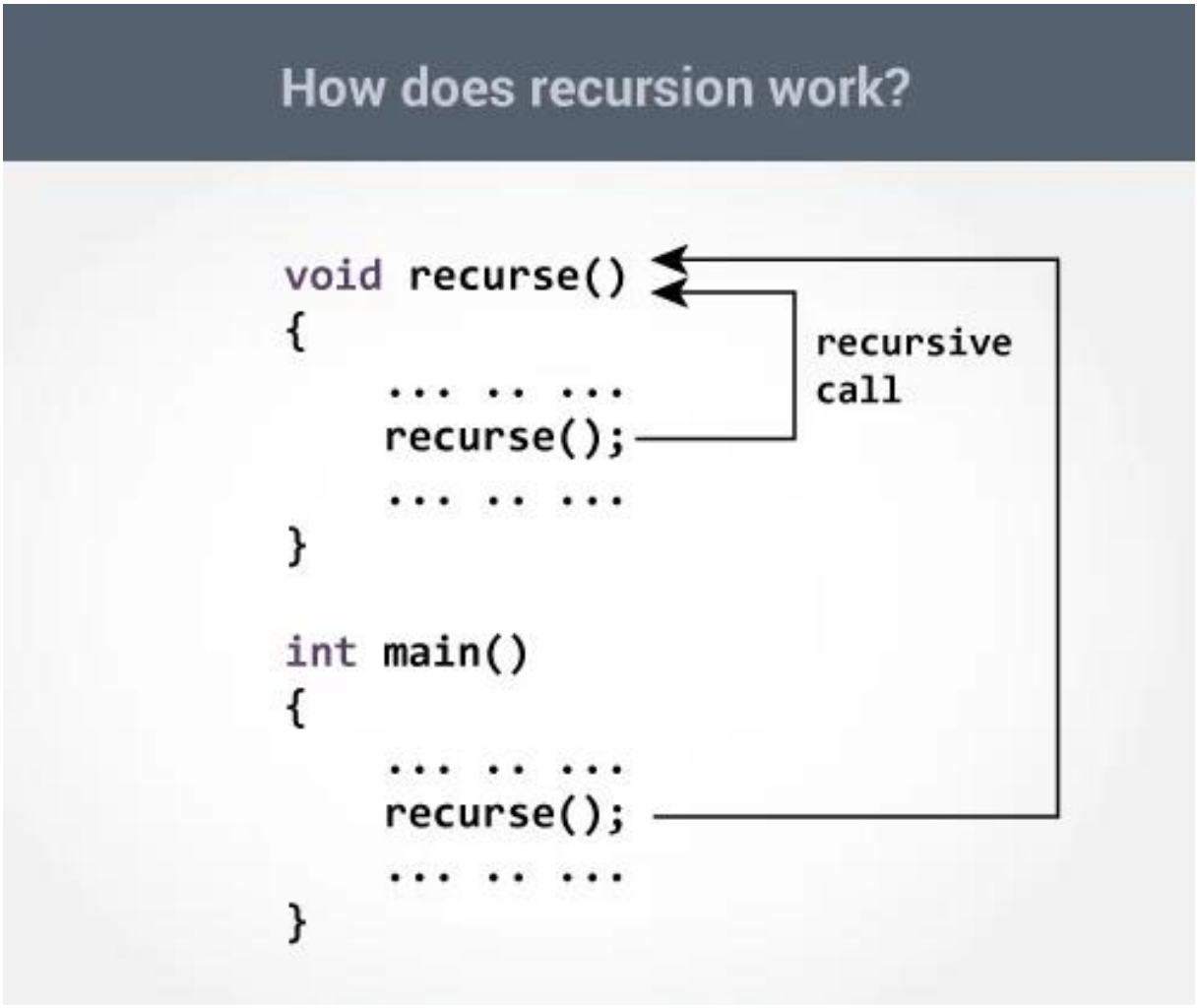
https://youtube.com/playlist?list=PLBlnK6fEyqRhwQbYrTDZYJaB4z1YgsAPW

https://www.scaler.com/topics/c/

Month#1

Resources for C - Recursion

Recursion is the process of repeating items in a self-similar way. In programming languages, if a program allows you to call a function inside the same function, then it is called a recursive call of the function.

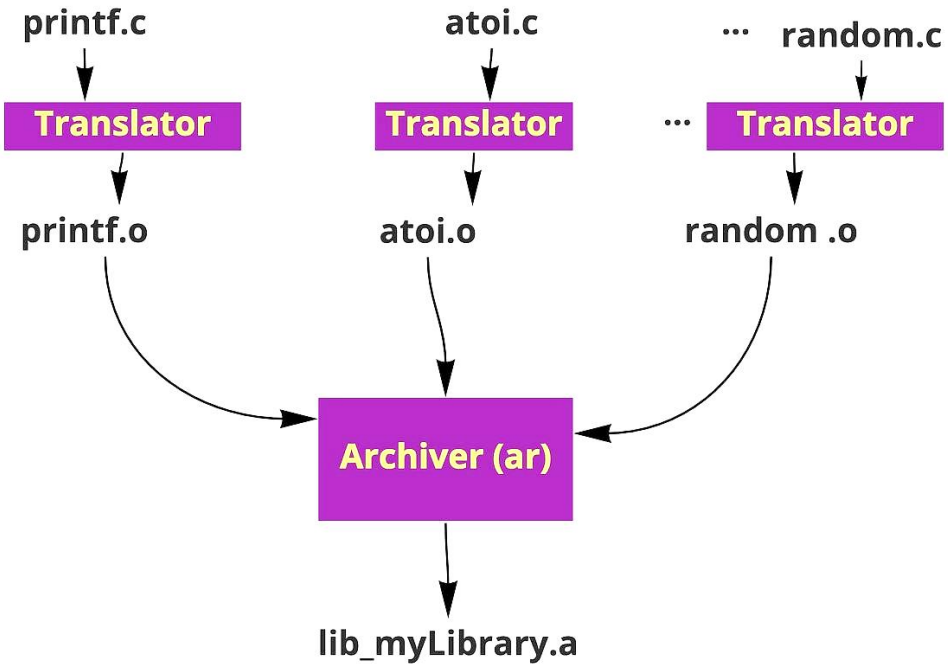


https://www.youtube.com/watch?v=IJDJ0kBx2LM&t=316s

https://www.youtube.com/watch?v=STWnc6ZY2fw

Resources for C - Static libraries

A static library is a file containing a collection of object files (*.o) that are linked into the program during the linking phase of compilation and are not relevant during runtime.

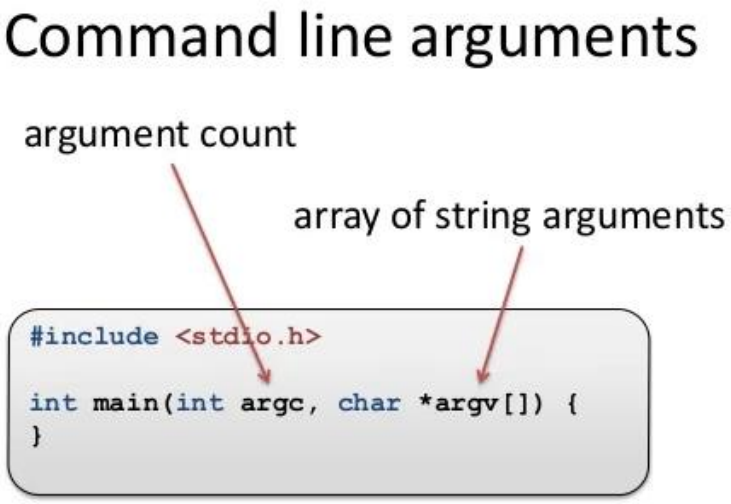


https://dev.to/iamkhalil42/all-you-need-to-know-about-c-static-libraries-1o0b

https://www.youtube.com/watch?v=3RmIVDgPmGk&t=6s

https://youtu.be/icbR8V5eOQc

Resources for C - argc, argv[]



argv[0] is the program name with full path
argv[1] is the first argument
argv[2] is the second argument, etc.

https://www.youtube.com/watch?v=wa2AfzyOff0

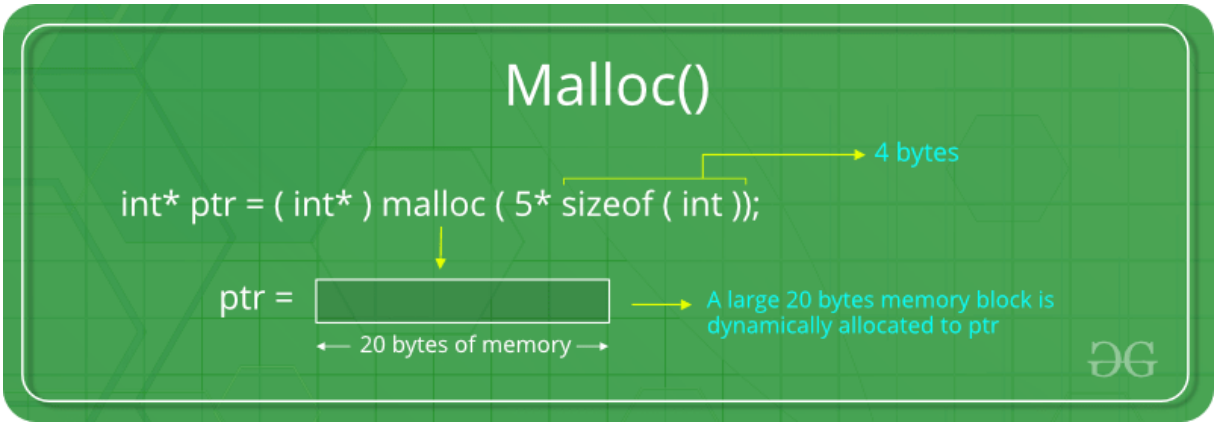
https://youtu.be/i2-I97Pc6O8

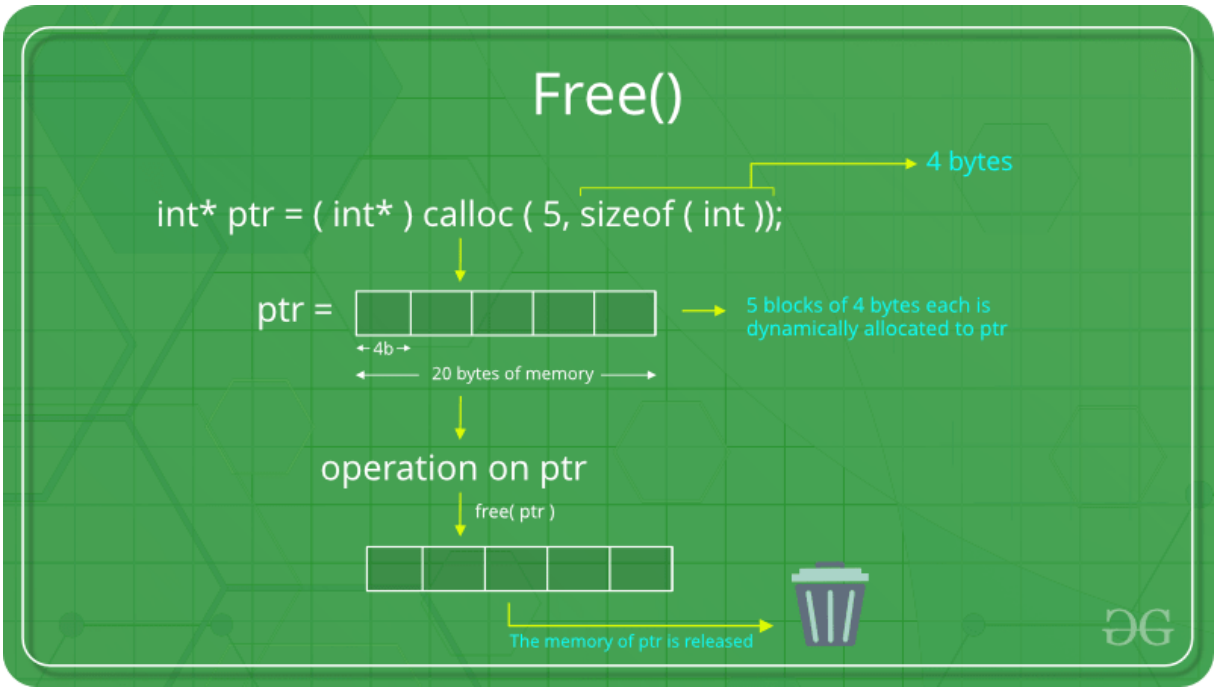
https://youtu.be/z6g_echs8jE

https://youtu.be/V2wrrFwKzsY

https://youtu.be/H0jWzoVU1kQ

Resources for C - malloc, free





https://www.youtube.com/watch?v=xDVC3wKjS64

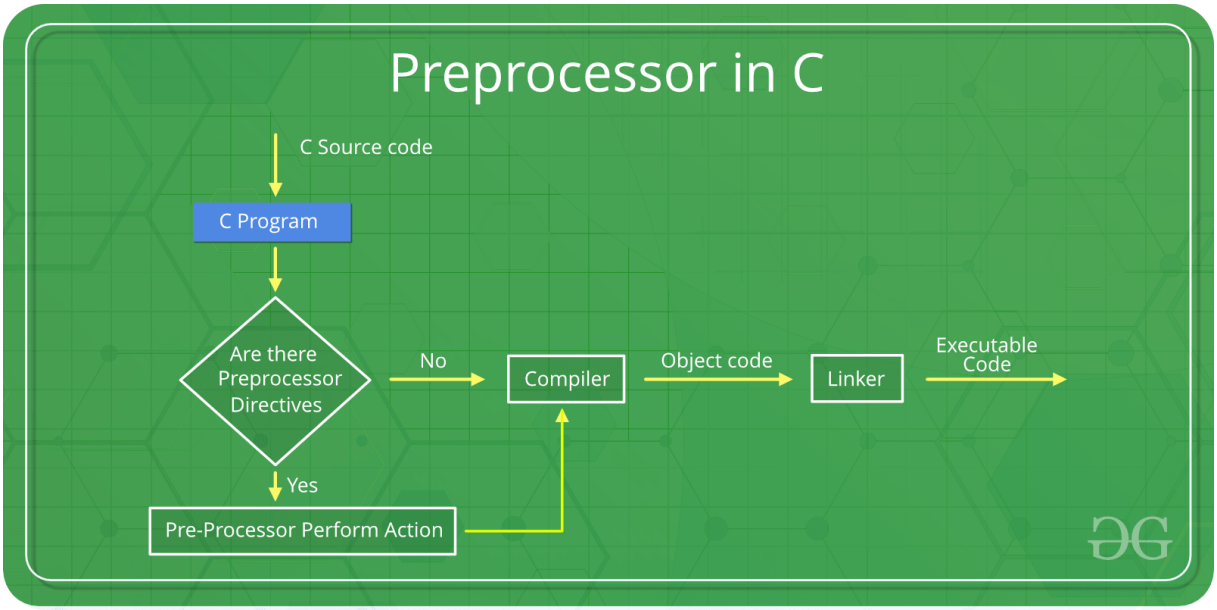
https://youtu.be/udfbq4M2Kfc

https://youtu.be/R0qIYWo8igs

https://youtu.be/v49bwqQ4ouM

https://www.youtube.com/watch?v=_8-ht2AKyH4

Resources for C- preprocessor



https://learnprogramo.com/what-is-preprocessor-in-c-34/

https://www.programiz.com/c-programming/c-preprocessor-macros

https://www.geeksforgeeks.org/preprocessor-works-c/

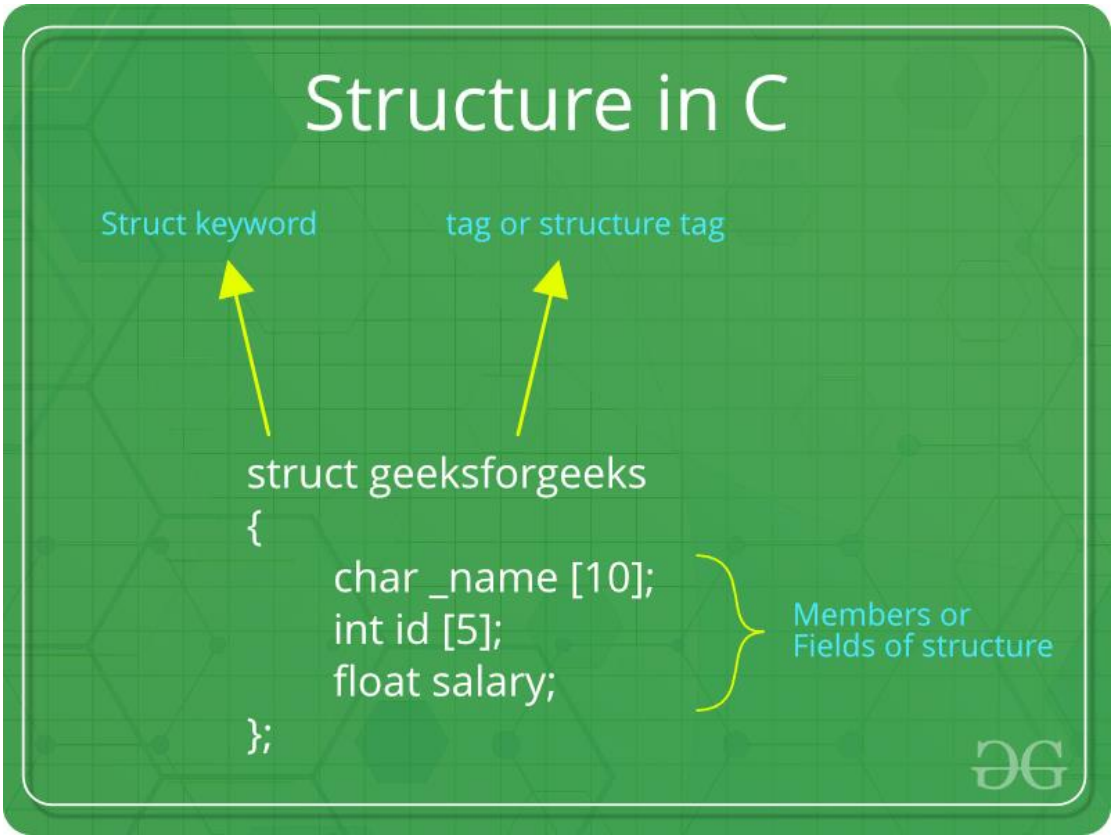
https://youtu.be/pw0uGPmGNl8

https://youtu.be/InxAy7eI_OE

https://youtu.be/F7dOF9E7QRo

https://youtu.be/feaVifwt7EI

Resources for C- Structure, Typedef




```
struct S {
    // ...
} typedef T;
```

https://learnprogramo.com/structures-and-unions-in-c-32/

https://www.youtube.com/watch?v=dqa0KMSMx2w

https://youtu.be/LpHnHRI6gLc

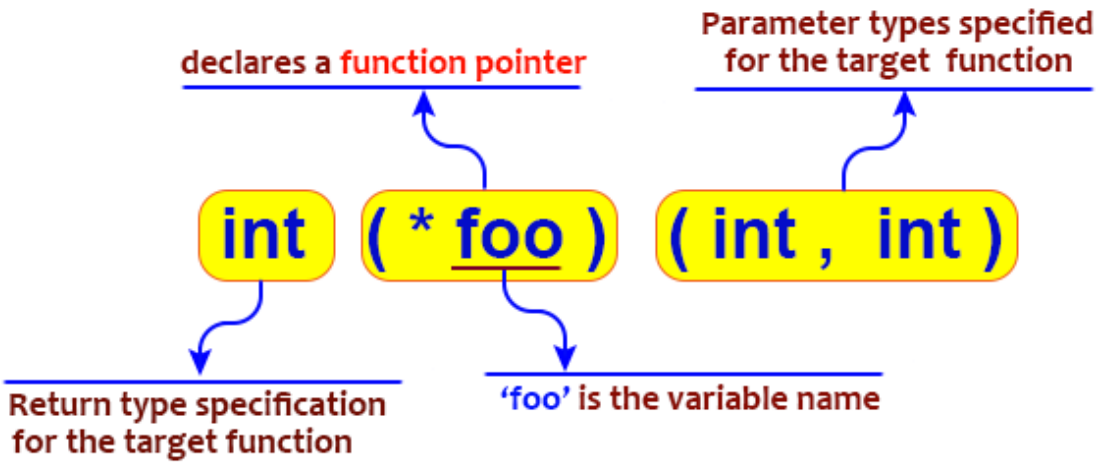
https://youtu.be/6Z-silgul48

https://youtu.be/OxkTCgGIWxk

https://youtu.be/ojhbmh5SJsw

https://youtube.com/playlist?list=PLBlnK6fEyqRiteqwlMLXYtZ16xXDR7MO0

Resources for C- Function Pointer



https://www.geeksforgeeks.org/function-pointer-in-c/

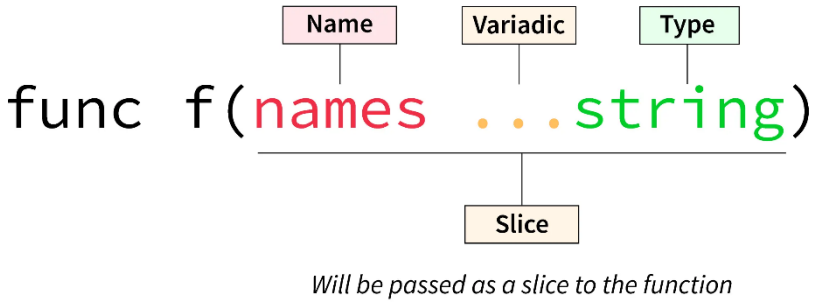
https://youtu.be/ynYtgGUNelE

https://www.youtube.com/watch?v=sxTFSDAZM8s&ab_channel=mycodeschool

https://youtu.be/qaszuaFXRTA

https://www.youtube.com/watch?v=wMI2k951nqs

Resources for C- Variadic Function



Will be passed as a slice to the function

https://www.youtube.com/watch?v=3iX9a_l9W9Y&ab_channel=PortfolioCourses

https://www.youtube.com/watch?v=S-ak715zIIE&ab_channel=JacobSorber

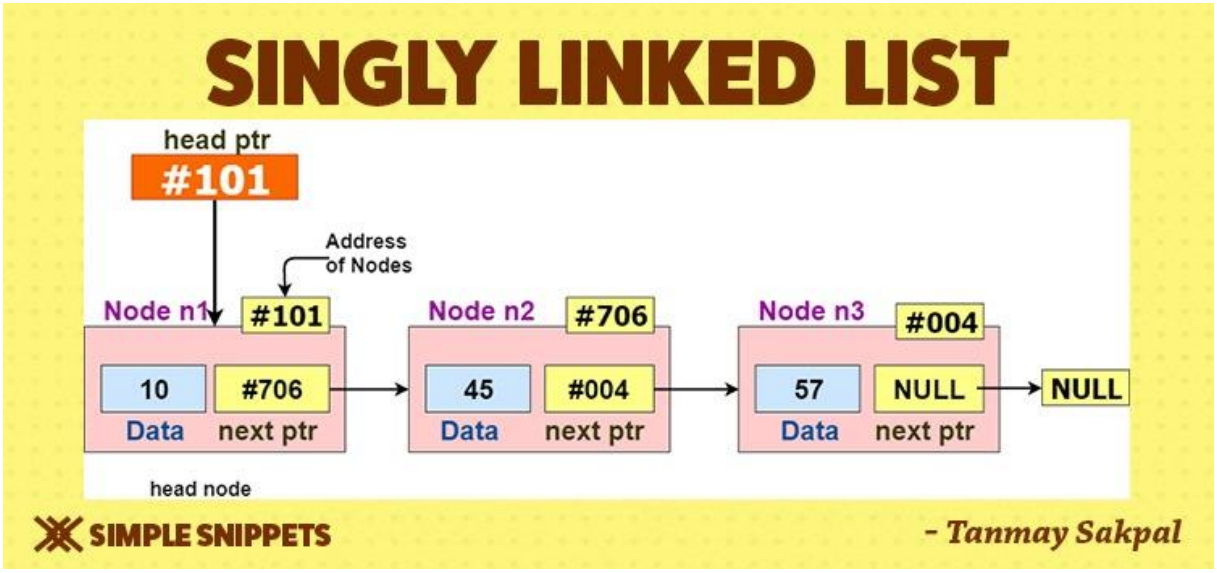
https://youtu.be/oDC208zvsdg

https://www.geeksforgeeks.org/variadic-functions-in-c/

https://medium.com/swlh/variadic-function-in-c-programming-d3632315a48e

https://youtu.be/Lh7xydr8zzU

Resources for C- Singly-linked lists



- **Articles**

https://medium.com/basecs/whats-a-linked-list-anyway-part-1-d8b7e6508b9d

https://medium.com/basecs/whats-a-linked-list-anyway-part-2-131d96f71996

https://www.geeksforgeeks.org/introduction-to-linked-list-data-structure-and-algorithm-tutorial/

https://www.geeksforgeeks.org/what-is-linked-list/

https://www.geeksforgeeks.org/what-is-linked-list/

- **videos**

https://www.youtube.com/playlist?list=PLBlnK6fEyqRi3-lvwLGzcaquOs5OBTCww

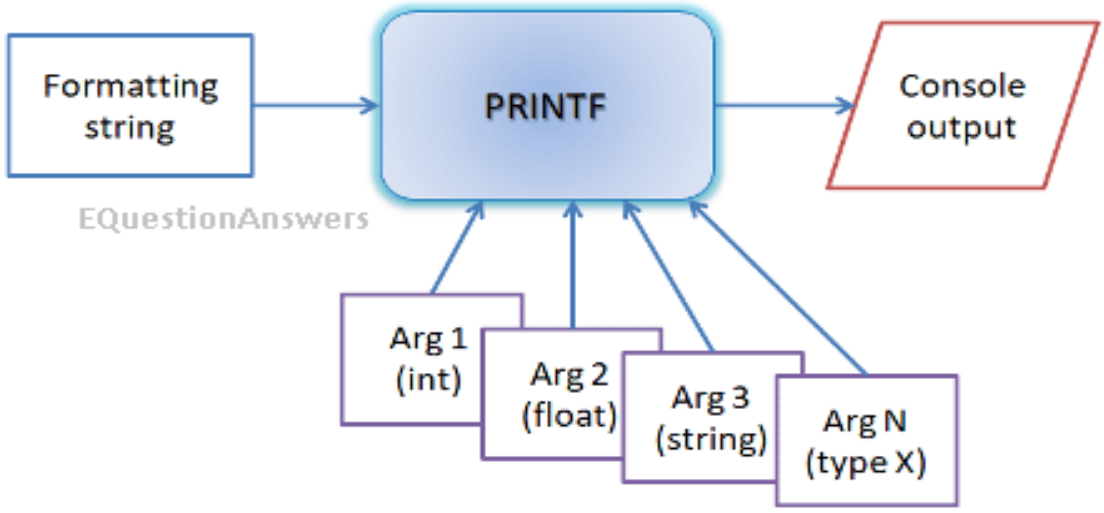
https://youtu.be/Hj_rA0dhr2I?t=678

https://www.youtube.com/watch?v=YjxKYxpf51E

- **Advanced_1 - Functions that are executed before**

https://www.tutorialspoint.com/functions-that-are-executed-before-and-after-main-in-c

Hello 👋 Printf 🐍



don't start printf if you don't have a fully understanding of how is it going on those topics.

C- Variadic Function

C- Structure, Typedef

C- Function Pointer

C - malloc, free

- **Articles**

this page seems like simple page but it helps us make coffee and read it slowly

https://www.lix.polytechnique.fr/~liberti/public/computing/prog/c/C/FUNCTIONS/format.html#period

https://www.beningo.com/7-steps-to-customizing-printf-with-a-circular-buffer/

https://scientyficworld.org/how-to-write-my-own-printf-in-c/

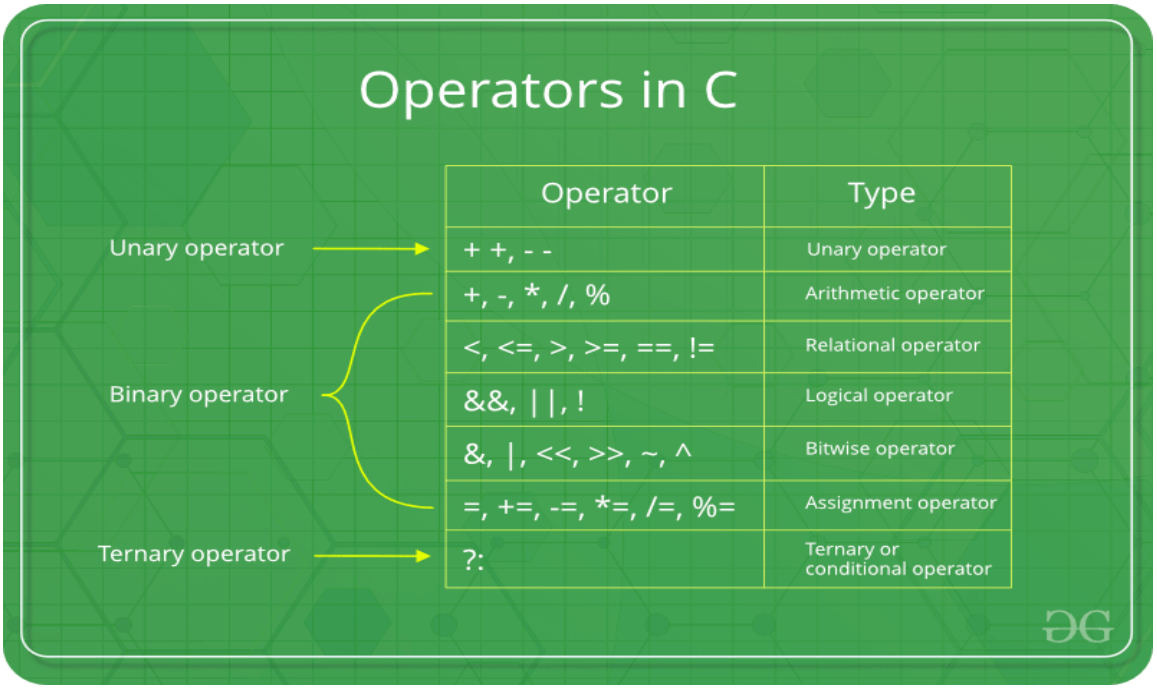
https://stackoverflow.com/questions/1735236/how-to-write-my-own-printf-in-c

- **Videos**

https://www.youtube.com/watch?v=3iX9a_l9W9Y&t=429s&ab_channel=PortfolioCourses

Month#2

Resources for C - Bit manipulation



- Use OR ('|') to set bits to '1'
- Use AND ('&') to set bits to 0
- Use XOR(^) to toggle bits (0 -> 1 and 1-> 0)

OR		
A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

AND		
A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

XOR		
A	B	Output
0	0	0
0	1	1
1	0	1
1	1	0

- **Articles:**

https://www.tutorialspoint.com/ansi_c/c_bits_manipulation.htm

https://nicolwk.medium.com/bitwise-operations-cheat-sheet-743e09aec5b5

- **Videos**

https://youtube.com/playlist?list=PL5DyztRVgtRUVORP3AXvX91uovcaZv0q9

https://youtu.be/7jkIUgLC29I

https://youtu.be/WBim3afbYQw

https://youtu.be/jlQmeyce65Q

- **Advanced**

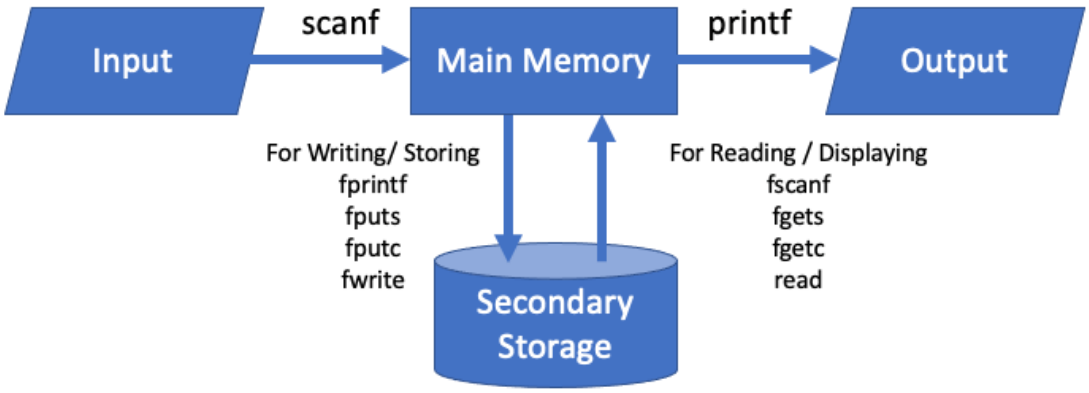
https://dev.to/ajipelumi/c-function-to-check-endianness-5d25

https://medium.com/@rickharris_dev/reverse-engineering-using-linux-gdb-a99611ab2d32

https://www.linkedin.com/pulse/crack-program-first-steps-lamine-bellilet/

https://hybridivy.wordpress.com/tag/hack/

Resources for C - File I/O



How do C programmers stay organized?

They always fclose the door after they fopen it :)

- **Articles**

File-IO Through Library Functions:

https://www.programiz.com/c-programming/c-file-input-output

https://www.geeksforgeeks.org/c-file-io/

A Little C Primer/C File-IO Through System Calls

https://www.geeksforgeeks.org/input-output-system-calls-c-create-open-close-read-write/

https://en.wikibooks.org/wiki/A_Little_C_Primer/C_File-IO_Through_System_Calls

File-IO Through Library Functions

https://www.youtube.com/watch?v=HQNsriyMhtY&pp=ygUNZmlsZSBpL28gaW4gYw==

File-IO Through System Calls

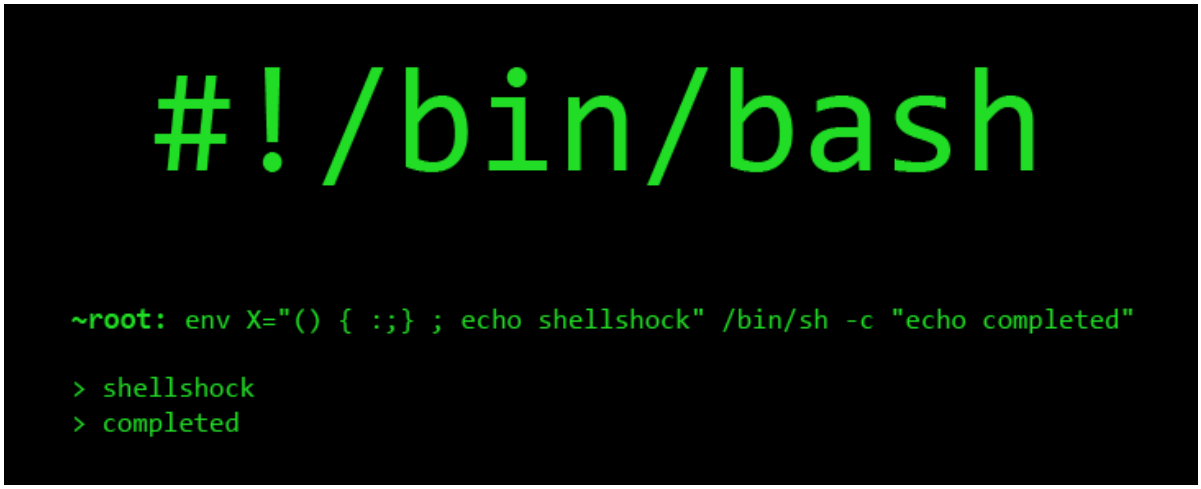
A simple explanation of the difference :

https://www.youtube.com/watch?v=BQJBe4IbsvQ&ab_channel=JacobSorber

A useful playlist for system calls (through videos 1 - 7) :

https://www.youtube.com/watch?v=M-Eo8QkG2gI

C - Simple Shell



These are a starting point to put you on the right path, as they help you, at first to get to know about the shell, what are the system calls and how to make your own shell.

>> We will update them once we get more helpful resources and get to the task knowing its requirements and what it needs, NEVER hesitate to share us your Resources if you have more helpful ones :).

What is Shell?

- **Articles**

https://www.geeksforgeeks.org/introduction-linux-shell-shell-scripting/

https://www.thegeekdiary.com/unix-linux-what-is-a-shell-what-are-different-shells/

https://www.geeksforgeeks.org/introduction-linux-shell-shell-scripting/

- **Videos**

https://www.youtube.com/watch?v=FpljIU9Z-f8&ab_channel=samit

❖System Calls

- **Articles**

https://www.geeksforgeeks.org/linux-system-call-in-detail/

https://www.cs.tufts.edu/comp/21/notes/processes/processes_c.shtml

https://perugini.cps.udayton.edu/teaching/books/SPUC/www/lecture_notes/processes.html

- **Videos**

https://www.youtube.com/watch?v=cex9XrZCU14&list=PLfqABt5AS4FkW5mOn2Tn9ZZLLDwA3kZUY&ab_channel=CodeVault

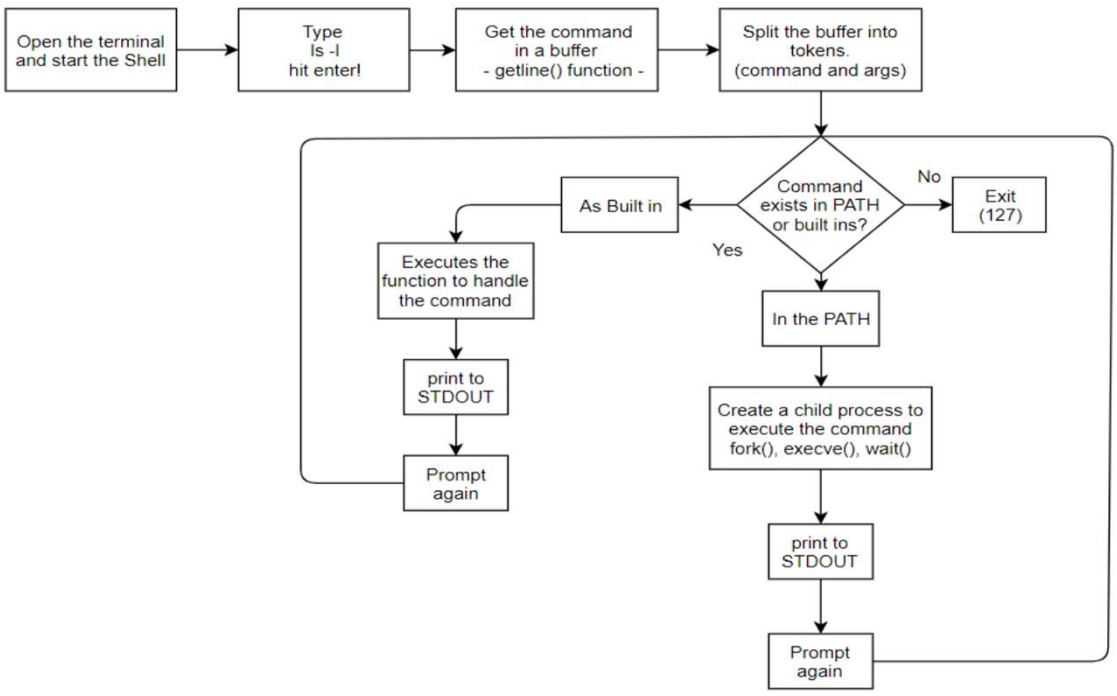
https://www.youtube.com/playlist?list=PL1XjRDnU2tOipNUtu22aHUGC4SADqHrYF

Shell Resources on A Daily Basis

We deeply Recommend to Not Copy or Depend on any written code, You have to understand what you need and figure the solutions urself and have your own code.

We are not responsible for anyone who would have a misusage of our help, you have to seek to be a Great SE not a full-mark getter.

❖Day 0



- **Articles**

https://www.linkedin.com/pulse/what-shell-how-works-daniela-g%C3%B3mez/

https://medium.com/@muxanz/how-the-shell-works-internally-when-entering-a-command-42f08458870

https://intranet.alxswe.com/concepts/64

- **Videos**

https://www.youtube.com/playlist?list=PL1XjRDnU2tOipNUtu22aHUGC4SADqHrYF

https://www.youtube.com/watch?v=nrO_pXGZc3Y

❖ Day 1

- **Articles**

https://www.baeldung.com/linux/pid-tid-ppid

You could find the getline( ) func in section 4 in this article:

https://www.studymite.com/blog/strings-in-c

https://www.geeksforgeeks.org/fork-system-call/

- **Videos**

A useful video, you could also check the whole playlist for relevant functions :

https://www.youtube.com/watch?v=tcYo6hipaSA&list=PLfqABt5AS4FkW5mOn2Tn9ZZLLDwA3kZUY&index=4

https://www.youtube.com/watch?v=iq7puCxsgHQ

https://www.youtube.com/watch?v=ss1-REMJ9GA

https://www.youtube.com/watch?v=kDxjcyHu_Qs

❖ Day 2

- **Articles**

https://medium.com/@winfredngenakilonzo/guide-to-code-a-simple-shell-in-c-bd4a3a4c41cd

https://www.geeksforgeeks.org/wait-system-call-c/

- **Videos**

https://www.youtube.com/watch?v=cex9XrZCU14&list=PLfqABt5AS4FkW5mOn2Tn9ZZLLDwA3kZUY&index=1

https://www.youtube.com/watch?v=wXkHx_noxws&list=PLxIRFba3rzLzxxZMMbrm_-mkI7mV9G0pj&index=5

❖ Day 3

- **Articles**

https://www.ics.uci.edu/~goodrich/teach/cs201P/notes/02_Environment_Variables.pdf

https://www.geeksforgeeks.org/environment-variables-in-linux-unix/

- **Videos**

Environment Variables (Arabic):

https://www.youtube.com/watch?v=lu7q8lbuUW4

Environment Variables:

https://www.youtube.com/watch?v=yM8v5i2Qjgg

https://www.youtube.com/watch?v=94URLRsjqMQ&list=PLfqABt5AS4FkW5mOn2Tn9ZZLLDwA3kZUY&index=5

OTHER RESOURCES

Tips to becoming a better coder

MAP YOUR MIND

A. PSEUDOCODE

1. What is pseudocode?
2. How to write pseudocode
3. Lecture on pseudocode

B. FLOWCHART

Visit Diagrams or Miro to create a flowchart

1. Flowchart
2. Create a flowchart
3. What is a flowchart? This is an in-depth video on a flowchart that explains it using C language.

GIT AND GITHUB

1. This is a quick explanation of Git and GitHub for beginners to help you know the basics.
2. Another practical explanation of Git and GitHub
3. This video uses visuals to explain Git in detail
4. Progit A detailed book that talks Git in details. Was worked on byt 100's of programmers

SHELL NAVIGATION

1. This is a quick crash course for the Command line.
2. This is a Linux series with a total of 108 videos. It talks about 100 Linux commands you should know and how to use them.
3. This is a video on Shell scripting that I highly recommend. You'd be amazed at what things you'd learn
4. All about learning the shell material

SHELL SCRIPTING VIDEOS AND MATERIALS

1. What is BASH scripting?
2. Writing BASH script
3. More on writing scripts
4. File permissions by guru99

5. All chmod commands
6. Keyboard shortcuts

SHELL VARIABLES AND EXPANSIONS

1. **Linux shell variables**
2. **Environment Variables**

SHELL I/O REDIRECTIONS

1. **Redirection in Linux**
2. **The three standard files in Linux**
3. **Pipes, Grep, Sort Commands**

MATERIALS TO READ ON SHELL COMMANDS, EMACS & VIM/VI

1. Linux command line
2. Basic Shell command
3. Git commands and another one
4. Bash command and make sure to read till the end if you want to see the other commands for files, processing, and networking
5. All about shell navigation
6. Emacs commands one and two
7. VI/Vim text editor

C PROGRAMMING LANGUAGE

0. Learn C in fantastic detail - This is at the top of my list.
1. An extensive and comprehensive read for C Programming by JavaTpoint
2. Another site to read for C by Educba
3. **C Programming For Beginners Theory** - A theoretical video on C
4. **C Programming for Beginners practical video**
Highly recommend this video for the practical aspect of c and learning to write mathematical problems
5. An interactive platform to learn to code in C, similar to Mimo and freecodecamp.
6. Short video on pointers and arrays
7. **C Interview**
8. **200+ C programs**

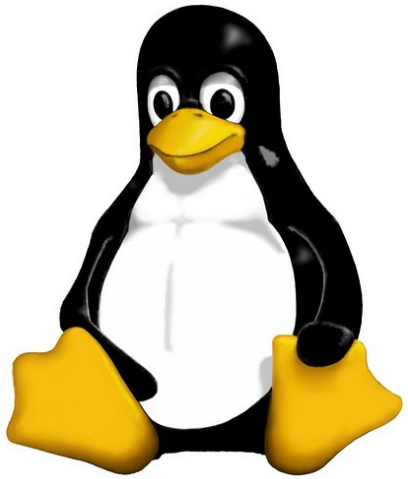
9. Books on C Programming Language - Google Drive (from Beginner to Advanced)
10. Programmiz interactive course on C
NOTE: DO NOT CLICK ON THE "INTERACTIVE C COURSE" unless you can afford to pay for the subscription and also want a certificate as well as want to do the projects given. If you have no need for those just start with **"C INTRODUCTION"**

CODING PLATFORMS

1. Online C compiler for practice also can be used for python, C++, SQL, and Java
2. Another online C compiler

freeworld.posterous.com

Linux Bash Shell Cheat Sheet



(works with about every distribution, except for apt-get which is Ubuntu/Debian exclusive)

Legend:

Everything in “<>” is to be replaced, ex: <fileName> --> iLovePeanuts.txt
Don't include the '=' in your commands
'..' means that more than one file can be affected with only one command ex: rm file.txt file2.txt movie.mov .. ..

Linux Bash Shell Cheat Sheet

Basic Commands

Basic Terminal Shortcuts

CTRL L = Clear the terminal
CTRL D = Logout
SHIFT Page Up/Down = Go up/down the terminal
CTRL A = Cursor to start of line
CTRL E = Cursor the end of line
CTRL U = Delete left of the cursor
CTRL K = Delete right of the cursor
CTRL W = Delete word on the left
CTRL Y = Paste (after CTRL U,K or W)
TAB = auto completion of file or command
CTRL R = reverse search history
!! = repeat last command
CTRL Z = stops the current command (resume with fg in foreground or bg in background)

Basic Terminal Navigation

ls -a = list all files and folders
ls <folderName> = list files in folder
ls -lh = Detailed list, Human readable
ls -l *.jpg = list jpeg files only
ls -lh <fileName> = Result for file only

cd <folderName> = change directory
if folder name has spaces use “ “
cd / = go to root
cd .. = go up one folder, tip: ../../../

du -h: Disk usage of folders, human readable
du -ah: “ “ “ files & folders, Human readable
du -sh: only show disc usage of folders

pwd = print working directory

man <command> = shows manual (RTFM)

Basic file manipulation

cat <fileName> = show content of file
(less, more)
head = from the top
-n <#oflines> <fileName>

tail = from the bottom
-n <#oflines> <fileName>

mkdir = create new folder
mkdir myStuff ..
mkdir myStuff/pictures/ ..

cp image.jpg newimage.jpg = copy and rename a file
cp image.jpg <folderName>/ = copy to folder
cp image.jpg folder/sameImageNewName.jpg
cp -R stuff otherStuff = copy and rename a folder
cp *.txt stuff/ = copy all of *<file type> to folder

mv file.txt Documents/ = move file to a folder
mv <folderName> <folderName2> = move folder in folder
mv filename.txt filename2.txt = rename file
mv <fileName> stuff/newfileName
mv <folderName>/ .. = move folder up in hierarchy

rm <fileName> .. = delete file (s)
rm -i <fileName> .. = ask for confirmation each file
rm -f <fileName> = force deletion of a file
rm -r <foldername>/ = delete folder

touch <fileName> = create or update a file

ln file1 file2 = physical link
ln -s file1 file2 = symbolic link

Linux Bash Shell Cheat Sheet

Basic Commands

Researching Files

The slow method (sometimes very slow):

```
locate <text> = search the content of all the files
locate <fileName> = search for a file
sudo updatedb = update database of files

find = the best file search tool (fast)
find -name “<fileName>”
find -name “text” = search for files who start with the word text
find -name “*text” = “ “ “ “ end “ “ “ “
```

Advanced Search:

```
Search from file Size (in ~)
      find ~ -size +10M = search files bigger than.. (M,K,G)

Search from last access
      find -name “<filetype>” -atime -5
            ('-' = less than, '+' = more than and nothing = exactly)

Search only files or directory’s
      find -type d --> ex: find /var/log -name "syslog" -type d
      find -type f = files

More info: man find, man locate
```

Extract, sort and filter data

```
grep <someText> <fileName> = search for text in file
      -i = Doesn't consider uppercase words
      -I = exclude binary files
grep -r <text> <folderName>/ = search for file names
      with occurrence of the text
```

With regular expressions:

```
grep -E ^<text> <fileName> = search start of lines
with the word text
grep -E <0-4> <fileName> =shows lines containing numbers 0-4
grep -E <a-zA-Z> <fileName> = retrieve all lines
with alphabetical letters

sort = sort the content of files
sort <fileName> = sort alphabetically
sort -o <file> <outputFile> = write result to a file
sort -r <fileName> = sort in reverse
sort -R <fileName> = sort randomly
sort -n <fileName> = sort numbers

wc = word count
wc <fileName> = nbr of line, nbr of words, byte size
      -l (lines), -w (words), -c (byte size), -m
      (number of characters)

cut = cut a part of a file
-c --> ex: cut -c 2-5 names.txt
      (cut the characters 2 to 5 of each line)
-d (delimiter)        (-d & -f good for .csv files)
-f (# of field to cut)

more info: man cut, man sort, man grep
```

Linux Bash Shell Cheat Sheet

Basic Commands

Time settings

date = view & modify time (on your computer)

View:
date “+%H” --> If it's 9 am, then it will show 09
date “+%H:%M:%Ss” = (hours, minutes, seconds)
%Y = years
Modify:
MMDDhhmmYYYY
Month | Day | Hours | Minutes | Year

sudo date 031423421997 = March 14th 1997, 23:42

Execute programs at another time

use 'at' to execute programs in the future

Step 1, write in the terminal: at <timeOfExecution> ENTER
ex --> **at 16:45** or **at 13:43 7/23/11** (to be more precise)
or after a certain delay:
at now +5 minutes (hours, days, weeks, months, years)
Step 2: <ENTER COMMAND> ENTER
repeat step 2 as many times you need
Step 3: CTRL D to close input

atq = show a list of jobs waiting to be executed

atrm = delete a job n°<x>
ex (delete job #42) --> atrm 42

sleep = pause between commands
with ';' you can chain commands, ex: touch file; rm file
you can make a pause between commands (**m**inutes, **h**ours, **d**ays)
ex --> touch file; sleep 10; rm file <-- 10 seconds

(continued)

crontab = execute a command regularly
-e = modify the crontab
-l = view current crontab
-r = delete you crontab
In crontab the syntax is
<Minutes> <Hours> <Day of month> <Day of week (0-6, 0 = Sunday)> <COMMAND>

ex, create the file movies.txt every day at 15:47:
47 15 * * * touch /home/bob/movies.txt
* * * * * --> every minute
at 5:30 in the morning, from the 1st to 15th each month:
30 5 1-15 * *
at midnight on Mondays, Wednesdays and Thursdays:
0 0 * * 1,3,4
every two hours:
0 */2 * * *
every 10 minutes Monday to Friday:
*/10 * * * 1-5

Execute programs in the background

Add a '&' at the end of a command
ex --> cp bigMovieFile.mp4 &

nohup: ignores the HUP signal when closing the console (process will still run if the terminal is closed)
ex --> nohup cp bigMovieFile.mp4

jobs = know what is running in the background

fg = put a background process to foreground
ex: fg (process 1), f%2 (process 2) f%3, ...

Linux Bash Shell Cheat Sheet

Basic Commands

Process Management

```
w = who is logged on and what they are doing

tload = graphic representation of system load average
       (quit with CTRL C)

ps = Static process list
       -ef --> ex: ps -ef | less
       -ejH --> show process hierarchy
       -u --> process's from current user

top = Dynamic process list
While in top:
```

- q to close top
- h to show the help
- k to kill a process

```
CTRL C to top a current terminal process

kill = kill a process
       You need the PID # of the process
              ps -u <AccountName> | grep <Application>
       Then
              kill <PID> .. .. ..
kill -9 <PID> = violent kill

killall = kill multiple process's
       ex --> killall locate

extras:
       sudo halt <-- to close computer
       sudo reboot <-- to reboot
```

Create and modify user accounts

```
sudo adduser bob = root creates new user
sudo passwd <AccountName> = change a user's password
sudo deluser <AccountName> = delete an account

addgroup friends = create a new user group
delgroup friends = delete a user group

usermod -g friends <Account> = add user to a group
usermod -g bob boby = change account name
usermod -aG friends bob = add groups to a user with-
out loosing the ones he's already in
```

File Permissions

```
chown = change the owner of a file
       ex --> chown bob hello.txt
chown user:bob report.txt = changes the user owning
report.txt to 'user' and the group owning it to 'bob'
-R = recursively affect all the sub folders
       ex --> chown -R bob:bob /home/Daniel

chmod = modify user access/permission – simple way
       u = user
       g = group
       o = other

       d = directory (if element is a directory)
       l = link (if element is a file link)
       r = read (read permissions)
       w = write (write permissions)
       x = eXecute (only useful for scripts and
       programs)
```

Linux Bash Shell Cheat Sheet

Basic Commands

File Permissions (continued)

```
'+' means add a right
'-' means delete a right
'=' means affect a right

ex --> chmod g+w someFile.txt
       (add to current group the right to modify someFile.txt)

more info: man chmod
```

Flow redirection

```
Redirect results of commands:

'>' at the end of a command to redirect the result to a file
       ex --> ps -ejH > process.txt
'>>' to redirect the result to the end of a file
```

Redirect errors:

```
'2>' at the end of the command to redirect the result to a file
       ex --> cut -d , -f 1 file.csv > file 2> errors.log
'2>&1' to redirect the errors the same way as the standard output

Read progressively from the keyboard

<Command> << <wordToTerminateInput>
       ex --> sort << END <-- This can be anything you want
              > Hello
              > Alex
              > Cinema
              > Game
              > Code
              > Ubuntu
              > END
```

Flow Redirection (continued)

```
terminal output:
Alex
Cinema
Code
Game
Ubuntu

       Another example --> wc -m << END
```

Chain commands

```
'|' at the end of a command to enter another one
       ex --> du | sort -nr | less
```

Archive and compress data

```
Archive and compress data the long way:

Step 1, put all the files you want to compress in
the same folder: ex --> mv *.txt folder/

Step 2, Create the tar file:
tar -cvf my_archive.tar folder/
       -c : creates a .tar archive
       -v : tells you what is happening (verbose)
       -f : assembles the archive into one file

       Step 3.1, create gzip file (most current):
gzip my_archive.tar
       to decompress: gunzip my_archive.tar.gz

Step 3.2, or create a bzip2 file (more powerful but slow):
bzip2 my_archive.tar
       to decompress: bunzip2 my_archive.tar.bz2
```

Linux Bash Shell Cheat Sheet

Basic Commands

Archive and compress data (continued)

step 4, to decompress the .tar file:
tar -xvf archive.tar archive.tar

Archive and compress data the fast way:

gzip: tar -zcvf my_archive.tar.gz folder/
decompress: tar -zcvf my_archive.tar.gz Documents/

bzip2: tar -jcvf my_archive.tar.gz folder/
decompress: tar -jxvf archive.tar.bz2 Documents/

Show the content of .tar, .gz or .bz2 without decompressing it:

gzip:
gzip -ztf archive.tar.gz
bzip2:
bzip2 -jtf archive.tar.bz2
tar:
tar -tf archive.tar

tar extra:
tar -rvf archive.tar file.txt = add a file to the .tar

You can also directly compress a single file and view the file without decompressing:

Step 1, use gzip or bzip2 to compress the file:
gzip numbers.txt

Step 2, view the file without decompressing it:
zcat = view the entire file in the console (same as cat)
zmore = view one screen at a time the content of the file (same as more)
zless = view one line of the file at a time (same as less)

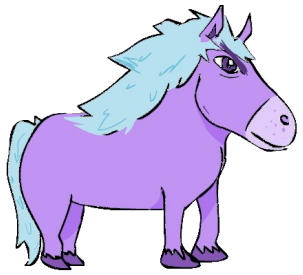
Installing software

When software is available in the repositories:
sudo apt-get install <nameOfSoftware>
ex--> sudo apt-get install aptitude

If you download it from the Internets in .gz format (or bz2) - “Compiling from source”
Step 1, create a folder to place the file:
mkdir /home/username/src <-- then cd to it

Step 2, with 'ls' verify that the file is there (if not, mv ../file.tar.gz /home/username/src/)

Step 3, decompress the file (if .zip: unzip <file>) <--
Step 4, use 'ls', you should see a new directory
Step 5, cd to the new directory
Step 6.1, use ls to verify you have an INSTALL file, then: more INSTALL
If you don't have an INSTALL file:
Step 6.2, execute **./configure** <-- creates a makefile
Step 6.2.1, run make <-- builds application binaries
Step 6.2.2 : switch to root --> su
Step 6.2.3 : make install <-- installs the software
Step 7, read the readme file



Handy Keyboard Shortcuts for the Linux Bash Terminal

Use these Linux Bash shortcuts for navigation, editing, command control, and easy access to history—all available in a free cheat sheet.

Bash Navigation	
Shortcut	**Action**
Ctrl + A	Move to the start of the command line
Ctrl + E	Move to the end of the command line
Ctrl + F	Move one character forward
Ctrl + B	Move one character backward
Ctrl + XX	Switch cursor position between start of the command line and the current position
Ctrl +] + x	Moves the cursor forward to next occurrence of x
Alt + F / Esc + F	Moves the cursor one word forward
Alt + B / Esc + B	Moves the cursor one word backward
Alt + Ctrl +] + x	Moves cursor to the previous occurrence of x

Bash Control/Process	
Shortcut	**Action**
Ctrl + L	Similar to clear command, clears the terminal screen
Ctrl + S	Stops command output to the screen
Ctrl + Z	Suspends current command execution and moves it to the background
Ctrl + Q	Resumes suspended command
Ctrl + C	Sends SIGI signal and kills currently executing command
Ctrl + D	Closes the current terminal

Bash History	
Shortcut	**Action**
Ctrl + R	Incremental reverse search of bash history
Alt + P	Non-incremental reverse search of bash history
Ctrl + J	Ends history search at current command
Ctrl + _	Undo previous command
Ctrl + P / Up arrow	Moves to previous command
Ctrl + N / Down arrow	Moves to next command
Ctrl + S	Gets the next most recent command
Ctrl + O	Runs and re-enters the command found via Ctrl + S and Ctrl + R
Ctrl + G	Exits history search mode
!!	Runs last command
!*	Runs previous command except its first word

Bash Editing	
Shortcut	**Action**
`Ctrl + U`	Deletes before the cursor until the start of the command
`Ctrl + K`	Deletes after the cursor until the end of the command
`Ctrl + W`	Removes the command/argument before the cursor
`Ctrl + D`	Removes the character under the cursor
`Ctrl + H`	Removes character before the cursor
`Alt + D`	Removes from the character until the end of the word
`Alt + Backspace`	Removes from the character until the start of the word
`Alt + . / Esc+.`	Uses last argument of previous command
`Alt + <`	Moves to the first line of the bash history
`Alt + >`	Moves to the last line of the bash history
`Esc + T`	Switch between last two words before cursor
`Alt + T`	Switches current word with the previous
Bash Information	
Shortcut	**Action**
`TAB`	Autocompletes the command or file/directory name
`~TAB TAB`	List all Linux users
`Ctrl + I`	Completes the command like TAB
`Alt + ?`	Display files/folders in the current path for help
`Alt + *`	Display files/folders in the current path as parameter

The Bash shell keyboard shortcuts work around the developer's DRY (Don't Repeat Yourself) philosophy. They help make effective use of your time by improving productivity in a fast-paced work environment.

#TALIS_ALX

By TALIS >>>>> Source

GitHub
GIT CHEAT SHEET

Git is the free and open source distributed version control system that's responsible for everything GitHub related that happens locally on your computer. This cheat sheet features the most important and commonly used Git commands for easy reference.

INSTALLATION & GUIS

With platform specific installers for Git, GitHub also provides the ease of staying up-to-date with the latest releases of the command line tool while providing a graphical user interface for day-to-day interaction, review, and repository synchronization.

GitHub for Windows
https://windows.github.com

GitHub for Mac
https://mac.github.com

For Linux and Solaris platforms, the latest release is available on the official Git web site.

Git for All Platforms
http://git-scm.com

SETUP

Configuring user information used across all local repositories

`git config --global user.name "[firstname lastname]"`
set a name that is identifiable for credit when review version history
`git config --global user.email "[valid-email]"`
set an email address that will be associated with each history marker
`git config --global color.ui auto`
set automatic command line coloring for Git for easy reviewing

SETUP & INIT

Configuring user information, initializing and cloning repositories

`git init`
initialize an existing directory as a Git repository
`git clone [url]`
retrieve an entire repository from a hosted location via URL

STAGE & SNAPSHOT

Working with snapshots and the Git staging area

`git status`
show modified files in working directory, staged for your next commit
`git add [file]`
add a file as it looks now to your next commit (stage)
`git reset [file]`
unstage a file while retaining the changes in working directory
`git diff`
diff of what is changed but not staged
`git diff --staged`
diff of what is staged but not yet committed
`git commit -m "[descriptive message]"`
commit your staged content as a new commit snapshot

BRANCH & MERGE

Isolating work in branches, changing context, and integrating changes

`git branch`
list your branches. a * will appear next to the currently active branch
`git branch [branch-name]`
create a new branch at the current commit
`git checkout`
switch to another branch and check it out into your working directory
`git merge [branch]`
merge the specified branch's history into the current one
`git log`
show all commits in the current branch's history

INSPECT & COMPARE

Examining logs, diffs and object information

`git log`	show the commit history for the currently active branch
`git log branchB..branchA`	show the commits on branchA that are not on branchB
`git log --follow [file]`	show the commits that changed file, even across renames
`git diff branchB...branchA`	show the diff of what is in branchA that is not in branchB
`git show [SHA]`	show any object in Git in human-readable format

SHARE & UPDATE

Retrieving updates from another repository and updating local repos

`git remote add [alias] [url]`	add a git URL as an alias
`git fetch [alias]`	fetch down all the branches from that Git remote
`git merge [alias]/[branch]`	merge a remote branch into your current branch to bring it up to date
`git push [alias] [branch]`	Transmit local branch commits to the remote repository branch
`git pull`	fetch and merge any commits from the tracking remote branch

TRACKING PATH CHANGES

Versioning file removes and path changes

`git rm [file]`	delete the file from project and stage the removal for commit
`git mv [existing-path] [new-path]`	change an existing file path and stage the move
`git log --stat -M`	show all commit logs with indication of any paths that moved

REWRITE HISTORY

Rewriting branches, updating commits and clearing history

`git rebase [branch]`	apply any commits of current branch ahead of specified one
`git reset --hard [commit]`	clear staging area, rewrite working tree from specified commit

IGNORING PATTERNS

Preventing unintentional staging or commiting of files

```
logs/
*.notes
pattern*/
```

Save a file with desired patterns as .gitignore with either direct string matches or wildcard globs.

`git config --global core.excludesfile [file]`	system wide ignore pattern for all local repositories

TEMPORARY COMMITS

Temporarily store modified, tracked files in order to change branches

`git stash`	Save modified and staged changes
`git stash list`	list stack-order of stashed file changes
`git stash pop`	write working from top of stash stack
`git stash drop`	discard the changes from top of stash stack

15-131 : Great Practical Ideas for Computer Scientists

VIM cheat sheet

Allison McKnight (aemcknig@andrew.cmu.edu)

Navigation

`h`	Move left	`H`	Top line on screen
`j`	Move down	`M`	Middle line on screen
`k`	Move up	`L`	Bottom line on screen
`l`	Move right		
`10j`	Move down 10 lines		
`gg`	First line of the file	`e`	The end of the current word
`G`	Last line of the file	`b`	Beginning of current word
`:20`	Line 20 of the file	`w`	Beginning of next word

`0`	Beginning of current line
`^`	First non-whitespace character of current line
`$`	End of current line
`%`	Move to matching parenthesis, bracket or brace

The left, right, up and down arrow keys can also be used to navigate.

Editing

`i`	Insert before current character
`a`	Insert after current character
`I`	Insert at the first non-whitespace character of the line
`o`	Insert a line below the current line, then enter insert mode
`O`	Insert a line above the current line, then enter insert mode
`r`	Overwrite one character and return to command mode
`U`	Undo
`Ctrl+R`	Redo

Opening, closing and saving files

`:w`	Save the current file
`:wq`	Save the current file and close it; exits vim if no open files remain
`:w newname`	Save a copy of the current file as 'newname,' but continue editing the original file
`:sav newname`	Save a copy of the current file as 'newname' and continue editing the file 'newname'
`:q!`	Close a file without saving
`:e somefile`	Opens `file` in the current buffer
`:x`	Write any changes to the file and close the file

Mode switching

`i`	Enter insert mode
`:`	Enter command mode
`R`	Enter replace mode
`v`	Enter visual mode (highlighting)
`V`	Enter visual line mode (highlighting lines)
`esc`	Return to normal mode from insert or replace mode
`esc+esc`	Return to normal mode from command or visual mode

Copy/pasting

Within vim

`y`	Yank
`c`	'Change'; cut and enter insert mode
`C`	Change the rest of the current line
`d`	Delete; cut but remain in normal mode
`D`	Delete the rest of the current line
`p`	Paste after the cursor
`P`	Paste before the cursor
`x`	Delete characters after the cursor
`X`	Delete characters before the cursor

Copy/paste commands operate on the specified range. If in visual mode, that range is the highlighted text. If in normal mode, that range is specified by a series of modifiers to the commands:

`cw`	Change one word
`c4w`	Change four words
`c4l`	Change four letters
`cc`	Change current line
`4x`	Change four characters after the cursor
`4p`	Paste five times after the cursor.

Modifiers work similarly for cut, delete, yank and paste.

From system clipboard

`:set paste`	Enter paste mode
`:set nopaste`	Exit paste mode
`Ctrl+Shift+V`	Paste into file, if in paste mode; `Command+Shift+V` for Mac

Replace

`:s/foo/bar/`	Replace the first occurrence of `foo` on the current line with `bar`
`:[range]s/foo/bar/[flags]`	Replace `foo` with `bar` in `range` according to `flags`

Ranges

`%`	The entire file
`'<,'>`	The current selection; the default range while in visual mode
`25`	Line 25
`25,50`	Lines 25-50
`$`	Last line; can be combined with other lines as in '50,$'
`.`	Current line; can be combined with other lines as in '.,50'
`,+2`	The current lines and the two lines therebelow
`-2,`	The current line and the two lines thereabove

Flags

`g`	Replace all occurrences on the specified line(s)
`i`	Ignore case
`c`	Confirm each substitution

Regular expressions

Both vim's find and replace functions accept regular expressions. By default, vim assumes that some characters are part of a regular expression and these characters must be escaped with '\' to be searched for literally; these characters include `(`, `)`, `*`, `.`, `^`, `$`. Other regular expression patterns are interpreted literally by default and must be escaped to be used as a part of a regular expression; these include `+`.

Search

`*`	Find the next instance of the current word
`#`	Find the previous instance of the current word
`n`	Find the next instance of the word being searched for, in the direction specified by the last use of {*,#}
`N`	Find the previous instance of the word being searched for, in the direction specified by the last use of {*,#}
`/word`	Find the next occurrence of 'word'
`/word\c`	Find the next occurrence of 'word', ignoring case ('\c' can appear anywhere in the sequence being searched for)
`/\<word\>`	Find the next occurrence of the word 'word', where 'word' is bounded by word boundaries (ex. space, dash)
`:noh`	Un-highlight words

Handy tricks

`~`	Toggle case of character beneath the cursor
`xp`	Transpose current character with that to its right
`J`	Join the current line and the line beneath it
`:Ni!`	Receive inquiry about shrubberies
`:help`	View vim help file
`:help foo`	View vim help entry on 'foo'

Resources (other than `google.com`)

`http://www.arl.wustl.edu/~fredk/Courses/Docs/vim/`
: The vim manual, online.

`http://www.stackoverflow.com/`
: Good general-purpose programming questions site.

`http://vim.wikia.com/`
: A wiki dedicated to vim.

Emacs Shortcut Cheatsheet

C-x means: hold Control and x at the same time
M-x means: type escape then x, or Meta and x

Content borrowed and updated (with permission) from Duane A. Bailey's guidelines from 2007.

Starting Emacs:		
start emacs	emacs	
Exiting Emacs:		
suspend emacs	C-z	
exit emacs	C-x C-c	
Files:		
read file	C-x C-f	
visit file other window	C-x C-v	
save file	C-x C-s	
insert file	C-x i	
write buffer to file	C-x C-w	
Getting Help:		
first time users	C-h t	
second time users	C-h ?	
help on keystroke	C-h k	
help on function	C-h f	
man page	M-x manual-entry	
Error recovery:		
abort command	C-g	
recover lost file	M-x recover-file	
undo	C-_	
restore buffer	M-x revert-buffer	
redraw screen	C-l	
Moving around:	Back	Forth
move by character	C-b	C-f
move by word	M-b	M-f
move by line	C-p	C-n
move by sentence	M-a	M-e
goto end of line	C-a	C-e
move by screen	M-v	C-v
top or bottom of buffer	C-x [	C-x]
center screen here	C-x l	
Marking:		
set mark	C-space	
exchange point and mark	C-x x	
mark buffer	C-x h	
Registers		
copy region to reg	C-x C-x	
get region from reg	C-x g	
Killing and Deleting:	Back	Forth
delete character	Delete	C-d
delete word	M-Del	M-d
delete rest of line	M-0 C-k	C-k
sent	C-x Del	M-k
region	C-w	
yank back (replace line cut)	C-y	
zap to <char>	M-z <char>	
Transpose		
characters	C-t	
words	M-t	
lines	C-x C-t	

Searching	
forward	C-s
backward	C-r
forward expression	C-M-s
backward expression	C-M-r
exit search	Return
undo last search char	Delete
abort search	C-g
Query Replace:	
start query replace	M-%
query replace word	C-u M-%
Once searching…	
replace & search	Space
replace & stay here	,
backup to previous	^
ignore and go on	Delete
replace all remaining	!
exit	Return
Multiple Open Windows:	
keep just this window	C-x 1
split window vertically	C-x 2
split window horizontal.	C-x 3
switch to a diff. window	C-x o
Buffers:	
switch to another buffer	C-x b
list all other buffers	C-x C-b
kill this buffer	C-x k
minibuffer	M-x
Within Minibuffer:	
complete command	Tab
show completions	?
complete and execute	Return
previous input	M-p
next input	M-n
abort	C-q
Keyboard Macros:	
start defining	C-x (
stop defining	C-x)
execute macro	C-x e
Compile something	
Compile window	M-x compile
Find next error	C-x '
Fun Stuff	
dungeon	M-x dunnet
tetris	M-x tetris
hide & seek	M-x blackbox
psychotherapy	M-x doctor
gomoku	M-x gomoku
robot game	M-x landmark
the snake game	M-x snake
peg solitare	M-x solitare

Quick and Dirty Guide to C

The single best book on C is The C Programming Language by Kernighan and Richie.

CODE:
Code for execution goes into files with ".c" suffix.
Shared decl's (included using #include "mylib.h") in "header" files, end in ".h"

COMMENTS:
Characters to the right of // are not interpreted; they're a comment.
Text between /* and */ (possibly across lines) is commented out.

DATA TYPES:

Name	Size	Description
char	1 byte	an ASCII value: e.g. 'a' (see: man ascii)
int/long	4 bytes	a signed integer: e.g. 97 or hex 0x61, oct 0x141
long long	8 bytes	a longer multi-byte signed integer
float	4 bytes	a floating-point (possibly fractional) value
double	8 bytes	a double length float

char, int, and double are most frequently and easily used in small programs
sizeof(double) computes the size of a double in addressable units (bytes)
Zero values represent logical false, nonzero values are logical true.
Math library (#include <math.h>, compile with -lm) prefers double.

CASTING:
Preceding a primitive expression with an alternate parenthesized type converts or "casts" value to a new value equivalent in new type:
```
  int a - (int) 3.131;    //assigns a=3 without complaint
```
Preceding any other expression with a cast forces new type for unchanged value.
```
  double b = 3.131;
  int a = *(int*)&b; //interprets the double b as an integer (not necessarily 3)
```

STRUCTS and ARRAYS and POINTERS and ADDRESS COMPUTATION:
Structs collect several fields into a single logical type:
```
  struct { int n; double root;} s; //s has two fields, n and root
  s.root = sqrt((s.n=7)); //ref fields (N.B. double parens=>assign OK!)
```
Arrays indicated by right associative brackets ([]) in the type declaration
```
  int a[10];   //a is a 10int array. a[0] is the first element. a[9] is the last
  char b[];    //in a function header, b is an array of chars with unknown length
  int c[2][3]; //c is an array of 2 arrays of three ints. a[1][0] follows a[0][2]
```
Array variables (e.g. a,b,c above) cannot be made to point to other arrays
Strings are represented as character arrays terminated by ASCII zero.
Pointers are indicated by left associative asterisk (*) in the type declarations:
```
  int a*a;     // a is a pointer to an integer
  char *b;     // b is a pointer to a character
  int *c[2];   // c is an array of two pointers to ints (same as int *(c[2]);
  int (*d)[2]; // d is a pointer to an array of 2 integers
```
Pointers are simply addresses. Pointer variables may be assigned.
Adding 1 computes pointer to the next value by adding sizeof(X) for type X
General int adds to pointer (even 0 or negative values) behave in the same way
Addresses may be computed with the ampersand (&) operator.
An array without an index or a struct without field computes its address:
```
  int a[10], b[20]; // two arrays
  int *p = a;       // p points to first int of array a
  p = b;            // p now points to the first int of array b
```
An array or pointer with an index n in square brackets returns the nth value:
```
  int a[10];        // an array
  int *p;
  int i = a[0];     // i is the first element of a
  i = *a;           // pointer dereference
  p = a;            // same as p = &a[0]
  p++;              // same as p = p+1; same as p=&a[1]; same as p = a+1
```
Bounds are not checked; your responsibility not to run off. Don't assume.
An arrow (-> no spaces!) dereferences a pointer to a field:
```
  struct { int n; double root; } s[1]; //s is pointer to struct or array of 1
  s->root = sqrt)s->n = 7); //s->root same as (*s).root or s[0].root
  printf("%g\n", s->root);
```

FUNCTIONS:
A function is a pointer to some code, parameterized by formal parameters, that may be executed by providing actual parameters. Functions must be declared before they are used, but code may be provided later. A sqrt function for positive n might be declared as:
```
  double sqrt(double n) {
     double guess;
     for (guess = n/2.0; abs(n-guess*guess)>0.001; guess = (n/guess+guess)/2);
     return guess;
  }
```
This function has type double (s*sqrt)(double).
```
  printf("%g\n", sqrt(7.0)); //calls sqrt; actuals are always passed by value
```
Functions parameters are always passed by value. Functions must return a value.
The return value need not be used. Function names with parameters returns the function pointer. Thus, an alias for sqrt may be declared:
```
  double (*root)(double) = sqrt;
  printf("%g\n", root(7.0));
```
Procedures or valueless functions return 'void'.
There must always be a main function that returns an int.
```
  int main(int argc, char **argv)   OR  int main(int argc, char *argv[])
```
Program arguments may be accessed as strings through main's array argv with argc elements. First is the program name. Function declarations are never nested.

OPERATIONS:

+, -, *, /, %	Arithmetic ops. /truncates on integers, % is remainder.
++i --i	Add or subtract 1 from i, assign result to i, return new val
i++ i--	Remember i, inc or decrement i, return remembered value
&& || !	Logical ops. Right side of && and || unless necessary
& | ^ ~	Bit logical ops: and, or, xor, complement.
>> <<	Shift right and left: int n=10; n <<2 computes 40.
=	Assignment is an operator. Result is value assigned.
+= -= *= etc	Perform binary op on left and right, assign result to left
== != < > <= >=	Comparison operators (useful only on primitive types)
?:	If-like expression: (x%2==0)?"even":"odd"
,	computing value is last: a, = b,c,d; exec's b,c,d then a=d

STATEMENTS:
Angle brackets identify syntactic elements and don't appear in real statements
```
  <expression> ;          //semicolon indicates end of a simple statement
  break;                  //quits the tightest loop or switch immediately
  continue;               //jumps to next loop test, skipping rest of loop body
  return x;               //quits this function, returns x as value
  { <statements> }        //curly-brace groups statements into 1 compound (no ;)
  if (<condition>) <stmt> //stmt executed if cond true (nonzero)
  if (<condition>) <stmt> else <stmt> // two-way condition
  while (<condition>) <stmt>    //repeatedly execute stmt only if condition true
  do <stmt> while (<condition>); //note the semicolon, executes at least once
  for (<init>; <condition>; <step>) <statement>

  switch (<expression>) {          //traditional "case statement"
    case <value>: <statement>      // this statement exec'd if val==expr
                  break;           // quit this when value == expression
    case <value2>: <statement2>    //executed if value2 = expression
    case <value3>: <statement3>    //executed if value3 = expression
                  break;           // quit
    default: <statement4>          // if matches no other value; may be first
                  break;           // optional (but encouraged) quit
  }
```

KEY WORDS

unsigned	before primitive type suggests unsigned operations
extern	in global declaration => symbol is for external use
static	in global declaration => symbol is local to this file
	in local decl'n => don't place on stack; keep value betw'n calls
typedef	before declaration defines a new type name, not a new variable

Quick and Dirty Guide to C

Content borrowed and updated (with permission) from Duane A. Bailey's guidelines from 2007.

I/O (#include <stdio.h>)

Default input comes from "stdin"; output goes to "stdout"; errors to "stderr".
Standard input and output routines are declared in stdio.h: #include <stdio.h>

Function	Description
fopen(name, "r")	opens file name for read, returns FILE *f; "w" allows write
fclose(f)	closes file f
getchar()	read 1 char from stdin or pushback; is EOF (int -1) if none
ungetch(c)	pushback char c into stdin for re-reading; don't change c
putchar(c)	write 1 char, c, to stdout
fgetc(f)	same as getchar(), but reads from file f
ungetc(c,f)	same as ungetchar() but onto file f
fputc(c,f)	same as putchar(c), but onto file f
fgets(s,n, f)	read string of n-1 chars to a s from f or til eof or \n
fputs(s,f)	writes string s to f: e.g. fputs("Hello world\n", stdout);
scanf(p,...)	reads ... args using format p (below); put &w/non-pointers
printf(p, ...)	write ... args using format p (below); pass args as is
fprintf(f,p,...)	same, but print to file f
fscanf(f,p,...)	same, but read from file f
sscanf(s,p,...)	same, but read from string s
sprintf(s,p,...)	same, as printf, but to string s
feof(f)	return true iff at end of file f

Formats use format characters preceded by escape %; other chars written as is>

char	meaning	char	meaning
%c	character	\n	newline (control-j)
%d	decimal integer	\t	tab (control-i)
%s	string	\\	slash
%g	general floating point	%%	perent

MEMORY (%include <stdlib.h>)

malloc(n)	alloc n bytes of memory; for type T: p = (T*)malloc(sizeof(t));
free(p)	free memory pointed at p; must have been alloc'd; don't re-free
calloc(n,s)	alloc n-array size s & clear; typ: a = (T*)calloc(n, sizeof(T));

MATH (#include <math.h> and link -lm; sometimes documented in man math)

All functions take and return double unless otherwise noted:

sin(a), cos(a), tan(a)	sine, cosine, tangent of double (in radians)
asine(y),acos(x),atan(r)	principle inverse of above
atan2(y,x)	principal inverse of tan(y/x) in same quadrant as (x,y)
sqrt(x)	root of x
log(x)	natural logarithm of x; others: log2(x) and log10(x)
exp(p)	e to the power of p; others: exp2(x) and exp10(x)
pow(x,y)	x to the power of y; like (expy*log(x))
ceil(x)	smallest integer (returned as double) no less than x
floor(x)	largest integer (returned as double) no greater than y

#include <stdlib.h> for these math functions

abs(x)	absolute value of x
random()	returns a random long
srandom(seed)	seeds the random generator with a new random seed

STRINGS (#include <string.h>)

strlen(s)	return length of string; number of characters before ASCII 0
strcpy(d,s)	copy string s to d and return d; N.B. parameter order like =
strncpy(d,s,n)	copy at most n characters of s to d and terminate; returns d
stpcpy(d,s)	like strcpy, but returns pointer to ASCII 0 terminarot in d
strcmp(s,t)	compare strings s and t and return first difference; 0=> equal
strncmp(s,t,n)	stop after at most n characters; needn't be null terminated
memcpy(d,s,n)	copy exactly n bytes from s to d; may fail if s overlaps d
memmove(d,s,n)	(slow) copy n bytes from s to d; won't fail if s overlaps d

COMPILING:

```
gcc prog.c     # compiles prog.c into a.out run result with ./a.out
gcc -o prog prog.c # compiles prog.c into prog; run result with ./prog
gcc -g -o prog prog.c # as above, but allows for debugging
```

A GOOD FIRST PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char** argv){
   printf("Hello, world.\n");
   return 0;
}
```

A WORD COUNT (WC)

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv){
   int charCount=0, wordCount=0, lineCount=0;
   int doChar=0, doWord=0, doLine=0, inWord = 0;
   int c;
   char *fileName = 0;
   FILE *f = stdin;
   while (argv++, --argc) {
       if (!strcmp(*argv,"-c")) doChar=1;
       else if (!strcmp(*argv,"-w")) doWord=1;
       else if (!strcmp(*argv,"-l")) doLine=1;
       else if (!(f = fopen((fileName = *argv), "r"))){
           printf("Usage: wc [-l] [-w] [-c]\n"); return 1;
       }
   }
   if (!(doChar || doWord || doLine)) doChar = doWord = doLine = 1;
   while (EOF != (c= fgetc(f))){
       charCount++;
       if (c == '\n') lineCount++;
       if (!iswpace(c)) {
         if (!inWord) { inWord = 1; wordcount++; }
       } else { inWord = 0; }
   }
   if (doLine) printf("%8d", lineCount);
   if (doWord) printf("%8d", wordCount);
   if (doChar) printf("%8d", charCount);
   if (fileName) printf(" %s", fileName);
   printf("\n");
 }
```

ADD YOUR NOTES HERE:

C Programming Cheat Sheet

By Jim Hall

C is a straightforward compiled programming language. Other programming languages borrow concepts from C, which makes C a great starting point if you want to learn programming languages such as Lua, C++, Java, or Go.

Basics

Include header files first, then define your global variables, then write your program.

```
/* comment to describe the program */
#include <stdio.h>
/* definitions */
int main(int argc, char **argv) {
   /* variable declarations */
   /* program statements */
}
```

Variables

Variable names can contain uppercase or lowercase letters (A to Z, or a to z), or numbers (0 to 9), or an underscore (_). Cannot start with a number.

`int`	Integer values (-1, 0, 1, 2, …)
`char`	Character values, such as letters
`float`	Floating point numbers (0.0, 1.1, 4.5, or 3.141)
`double`	Double precision numbers, like float but bigger

Functions

Indicate the function type and name followed by variables inside parentheses. Put your function statements inside curly braces.

```
int celsius(int fahr) {
  int cel;
  cel = (fahr - 32) * 5 / 9;
  return cel;
}
```

Allocate memory with **malloc**. Resize with **realloc**. Use **free** to release.

```
int *array;
int *newarray;

arr = (int *) malloc(sizeof(int) * 10);
if (arr == NULL) {
   /* fail */
}

newarray = (int *) realloc(array,
sizeof(int) * 20);

if (newarray == NULL) {
   /* fail */
}
arr = newarray;

free(arr);
```

C Programming Cheat Sheet

By Jim Hall

Binary operators

`a & b`	Bitwise AND (1 if both bits are 1)
`a | b`	Bitwise OR (1 if either bits are 1)
`a ^ b`	Bitwise XOR (1 if bits differ)
`a<<n`	Shift bits to the left
`a>>n`	Shift bits to the right

Assignment shortcuts

`a += b;`	Addition	`a = a + b;`
`a -= b;`	Subtraction	`a = a - b;`
`a *= b;`	Multiplication	`a = a * b;`
`a /= b;`	Division	`a = a / b;`
`a %= b;`	Modulo	`a = a % b;`

Useful functions <stdio.h>

`stdin`	Standard input (from user or another program)
`stdout`	Standard output (print)
`stderr`	Dedicated error output

```
FILE *fopen(char *filename, char *mode);

size_t fread(void *ptr, size_t size,
size_t nitems, FILE *stream);
int fclose(FILE *stream);

int puts(char *string);
int printf(char *format, …);
int fprintf(FILE *stream, char *format);
int sprintf(char *string, char *format);

int getc(FILE *stream);
int putc(int ch, FILE *stream);

int getchar();
int putchar(int ch);
```

Useful functions <stdlib.h>

```
void *malloc(size_t size);
void *realloc(void *ptr, size_t newsize);

void free(void *ptr);

void qsort(void *array, size_t nitems,
size_t size, int (*compar)(void *a, void
*b));

void *bsearch(void *key, void *array,
size_t nitems, size_t size, int (*compar)
(void *a, void *b));

void srand(unsigned int seed);

void rand();
```

Always test for **NULL** when allocating memory with ***malloc*** or ***realloc***.

If you ***malloc*** or ***realloc***, you should also ***free***. But only free memory once.

Programme Structure

FOUNDATIONS(9 Months)

Low Level

Understand programming languages and Unix system work or “what is going on under the hood”.

TECHNOLOGIES
-C programming
-Graphic programming
-Unix programming
-Data Structures & algorithms
-Reverse engineering & security

High Level

Create a complete web service from databases, back-end and an amazing front-end website with the latest technologies.

TECHNOLOGIES
- Python
- Front-end
- Object Oriented programming
- Databases

DevOps

Create a complete web service from databases, back-end and an amazing front-end website with the latest technologies.

TECHNOLOGIES
-C programming
-Graphic programming
-Unix programming
-Data Structures & algorithms
-Reverse engineering & security

SPECIALIZATION (3 Months)

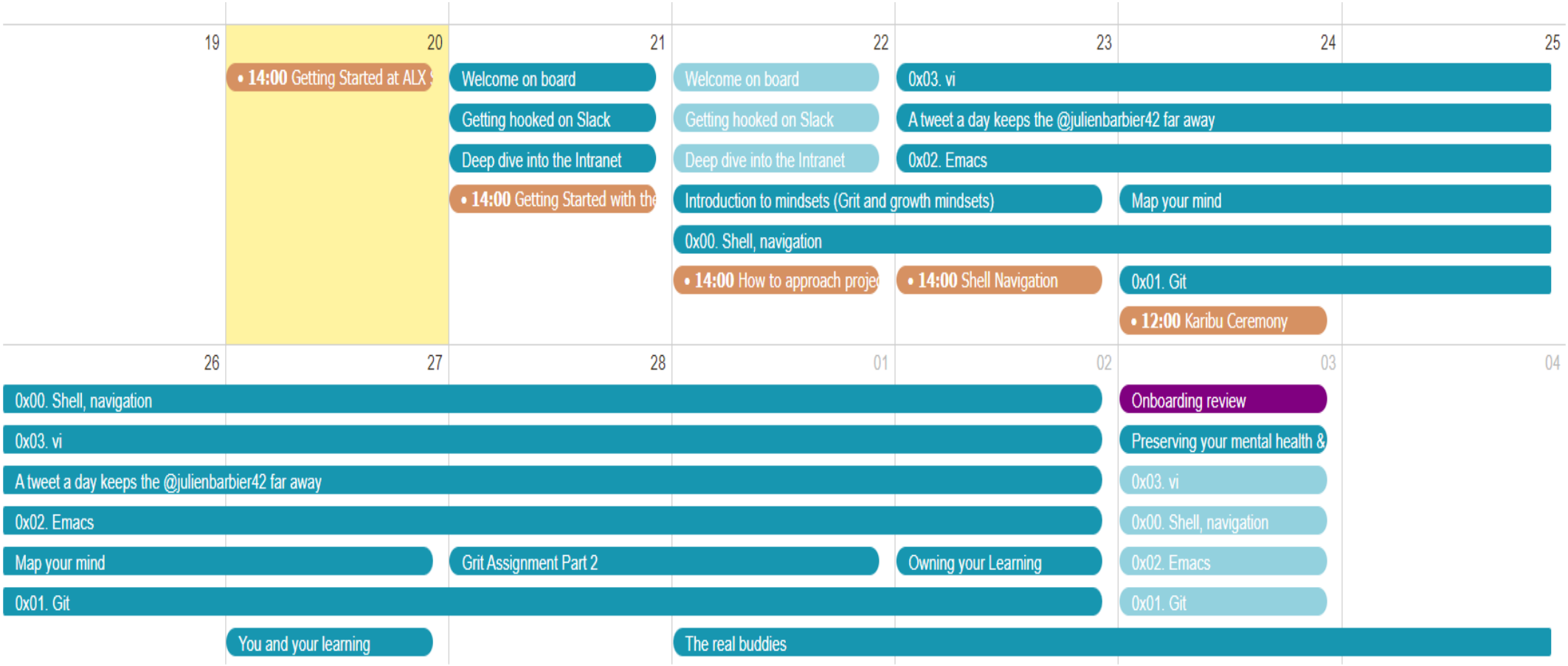
Specialization

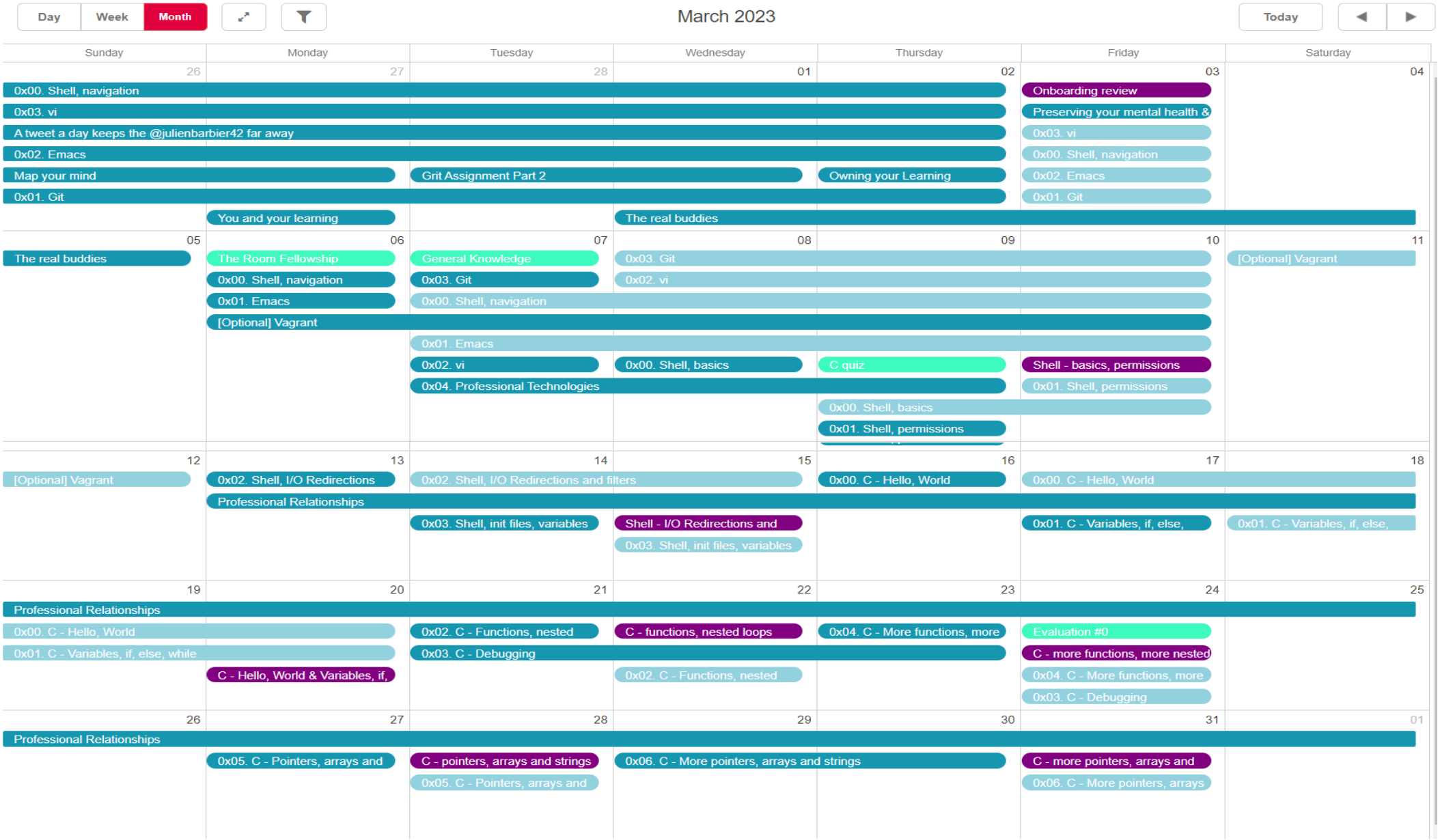
Create a complete web service from databases, back-end and an amazing front-end website with the latest technologies.

TECHNOLOGIES
-C programming
-Graphic programming
-Unix programming
-Data Structures & algorithms
-Reverse engineering & security

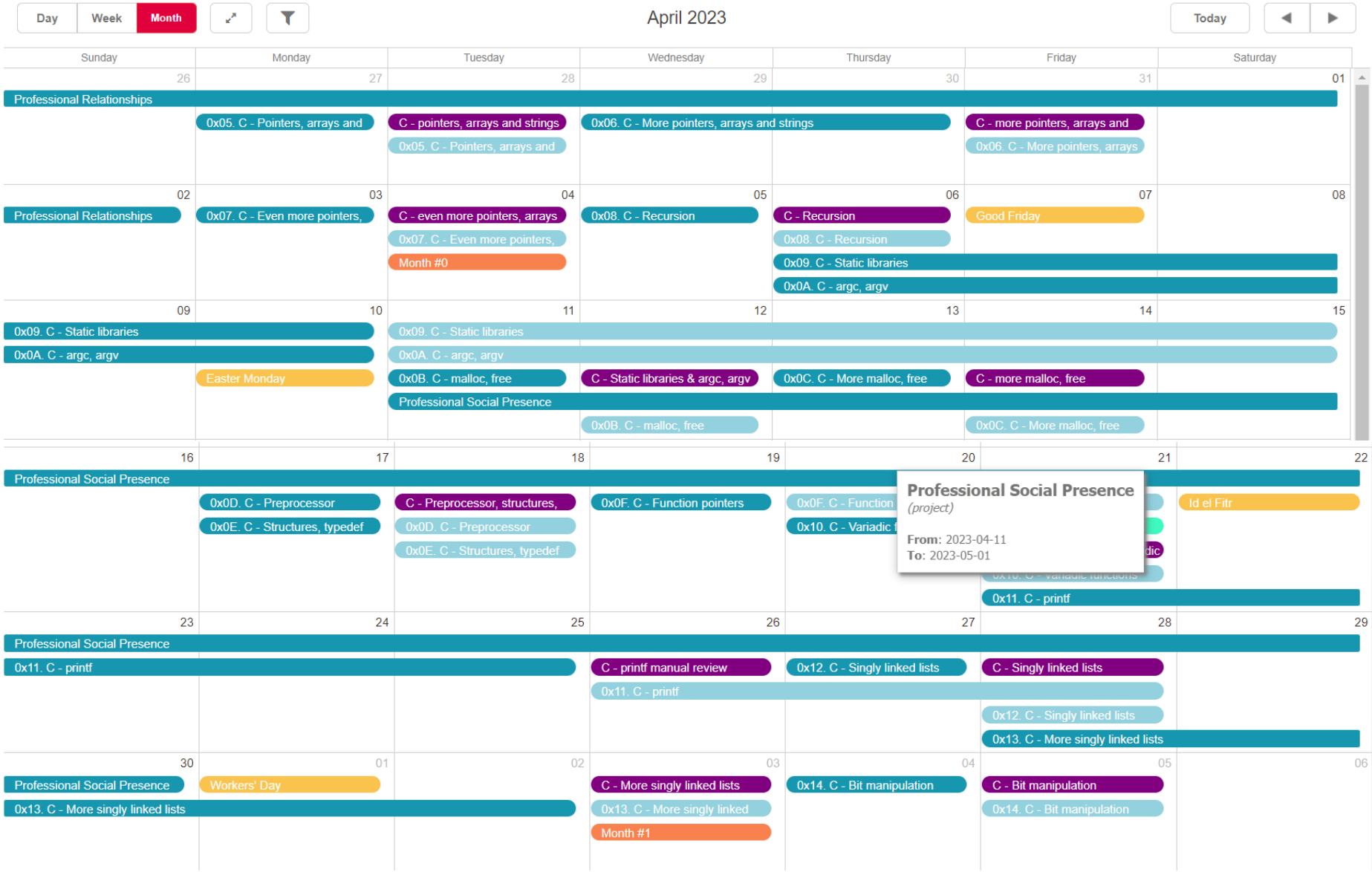
These are foundation languages you need to launch your career in software technology, we have carefully mapped out these technologies in terms of their level of complexity.

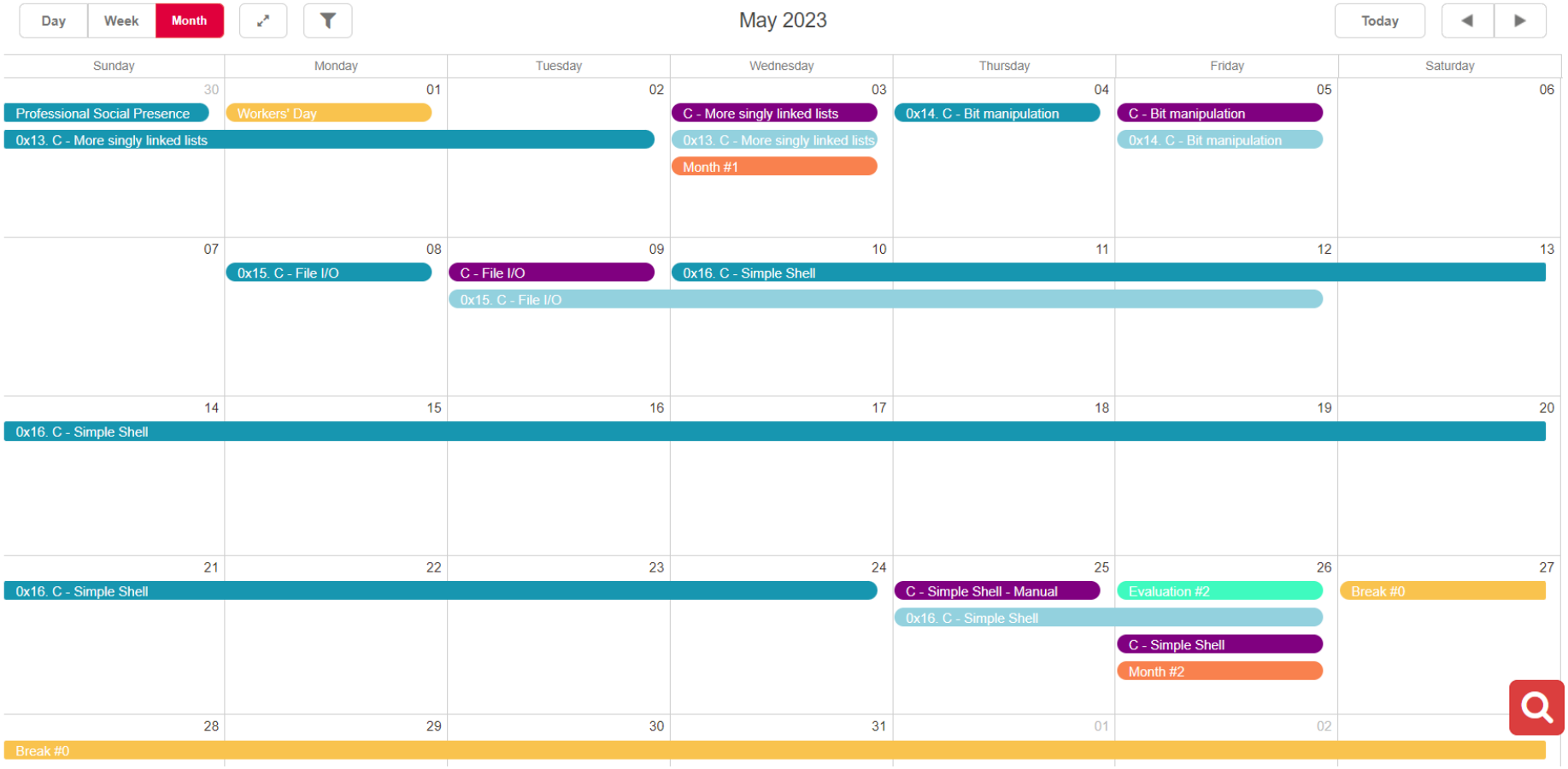
Planner

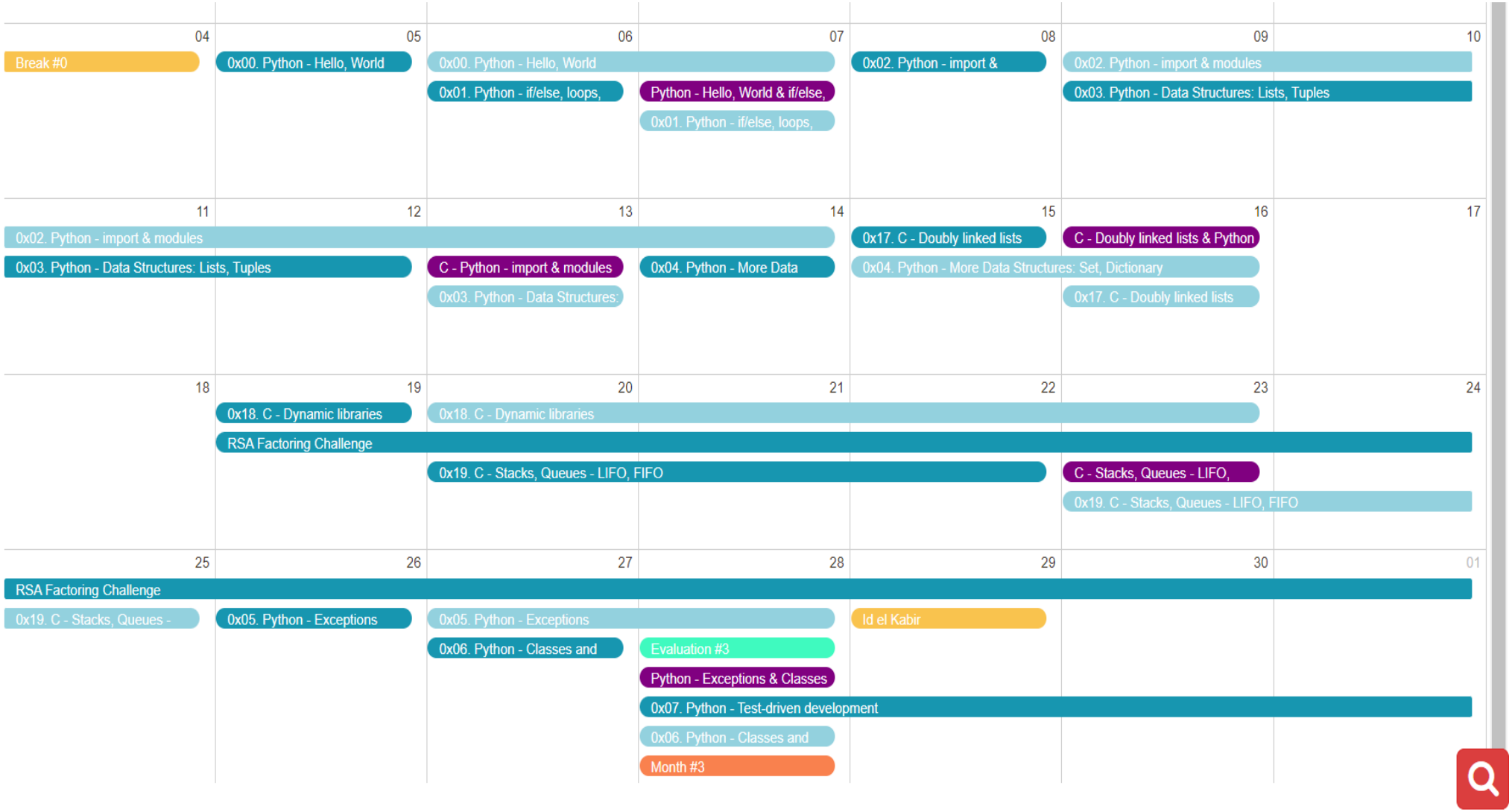

19
20
21
22
23
24
25
• 14:00 Getting Started at ALX
Welcome on board
Welcome on board
0x03. vi
Getting hooked on Slack
Getting hooked on Slack
A tweet a day keeps the @julienbarbier42 far away
Deep dive into the Intranet
Deep dive into the Intranet
0x02. Emacs
• 14:00 Getting Started with the
Introduction to mindsets (Grit and growth mindsets)
Map your mind
0x00. Shell, navigation
• 14:00 How to approach projec
• 14:00 Shell Navigation
0x01. Git
• 12:00 Karibu Ceremony
26
27
28
01
02
03
04
0x00. Shell, navigation
Onboarding review
0x03. vi
Preserving your mental health &
A tweet a day keeps the @julienbarbier42 far away
0x03. vi
0x02. Emacs
0x00. Shell, navigation
Map your mind
Grit Assignment Part 2
Owning your Learning
0x02. Emacs
0x01. Git
0x01. Git
You and your learning
The real buddies


Day
Week
Month
March 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
26
27
28
01
02
03
04
0x00. Shell, navigation
0x03. vi
A tweet a day keeps the @julienbarbier42 far away
0x02. Emacs
Map your mind
Grit Assignment Part 2
Owning your Learning
0x01. Git
You and your learning
The real buddies
Onboarding review
Preserving your mental health &
0x03. vi
0x00. Shell, navigation
0x02. Emacs
0x01. Git
05
06
07
08
09
10
11
The real buddies
The Room Fellowship
General Knowledge
0x03. Git
0x00. Shell, navigation
0x03. Git
0x02. vi
0x01. Emacs
0x00. Shell, navigation
[Optional] Vagrant
0x01. Emacs
0x02. vi
0x00. Shell, basics
C quiz
0x04. Professional Technologies
0x00. Shell, basics
0x01. Shell, permissions
[Optional] Vagrant
Shell - basics, permissions
0x01. Shell, permissions
[Optional] Vagrant
12
13
14
15
16
17
18
[Optional] Vagrant
0x02. Shell, I/O Redirections
0x02. Shell, I/O Redirections and filters
0x00. C - Hello, World
0x00. C - Hello, World
Professional Relationships
0x03. Shell, init files, variables
Shell - I/O Redirections and
0x03. Shell, init files, variables
0x01. C - Variables, if, else,
0x01. C - Variables, if, else,
19
20
21
22
23
24
25
Professional Relationships
0x00. C - Hello, World
0x01. C - Variables, if, else, while
C - Hello, World & Variables, if,
0x02. C - Functions, nested
0x03. C - Debugging
C - functions, nested loops
0x02. C - Functions, nested
0x04. C - More functions, more
Evaluation #0
C - more functions, more nested
0x04. C - More functions, more
0x03. C - Debugging
26
27
28
29
30
31
01
Professional Relationships
0x05. C - Pointers, arrays and
C - pointers, arrays and strings
0x05. C - Pointers, arrays and
0x06. C - More pointers, arrays and strings
C - more pointers, arrays and
0x06. C - More pointers, arrays

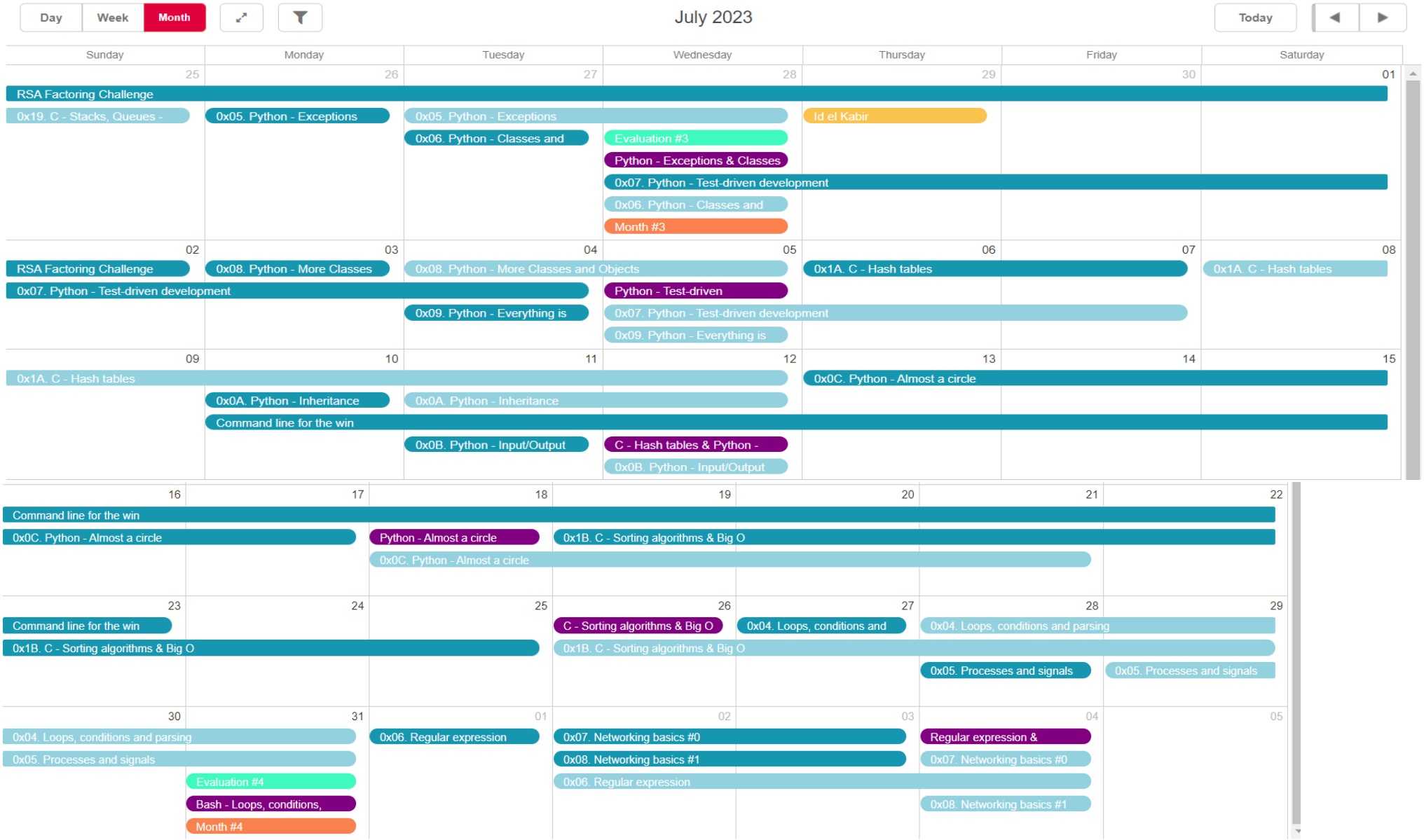
My planning


Day
Week
Month
April 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
26 27 28 29 30 31 01
Professional Relationships
0x05. C - Pointers, arrays and
C - pointers, arrays and strings
0x06. C - More pointers, arrays and strings
C - more pointers, arrays and
0x05. C - Pointers, arrays and
0x06. C - More pointers, arrays
02 03 04 05 06 07 08
Professional Relationships
0x07. C - Even more pointers,
C - even more pointers, arrays
0x08. C - Recursion
C - Recursion
Good Friday
0x07. C - Even more pointers,
0x08. C - Recursion
Month #0
0x09. C - Static libraries
0x0A. C - argc, argv
09 10 11 12 13 14 15
0x09. C - Static libraries
0x09. C - Static libraries
0x0A. C - argc, argv
0x0A. C - argc, argv
Easter Monday
0x0B. C - malloc, free
C - Static libraries & argc, argv
0x0C. C - More malloc, free
C - more malloc, free
Professional Social Presence
0x0B. C - malloc, free
0x0C. C - More malloc, free
16 17 18 19 20 21 22
Professional Social Presence
0x0D. C - Preprocessor
C - Preprocessor, structures,
0x0F. C - Function pointers
0x0F. C - Function
Id el Fitr
0x0E. C - Structures, typedef
0x0D. C - Preprocessor
0x10. C - Variadic
0x0E. C - Structures, typedef
Professional Social Presence
(project)
From: 2023-04-11
To: 2023-05-01
0x11. C - printf
23 24 25 26 27 28 29
Professional Social Presence
0x11. C - printf
C - printf manual review
0x12. C - Singly linked lists
C - Singly linked lists
0x11. C - printf
0x12. C - Singly linked lists
0x13. C - More singly linked lists
30 01 02 03 04 05 06
Professional Social Presence
Workers' Day
C - More singly linked lists
0x14. C - Bit manipulation
C - Bit manipulation
0x13. C - More singly linked lists
0x13. C - More singly linked
0x14. C - Bit manipulation
Month #1

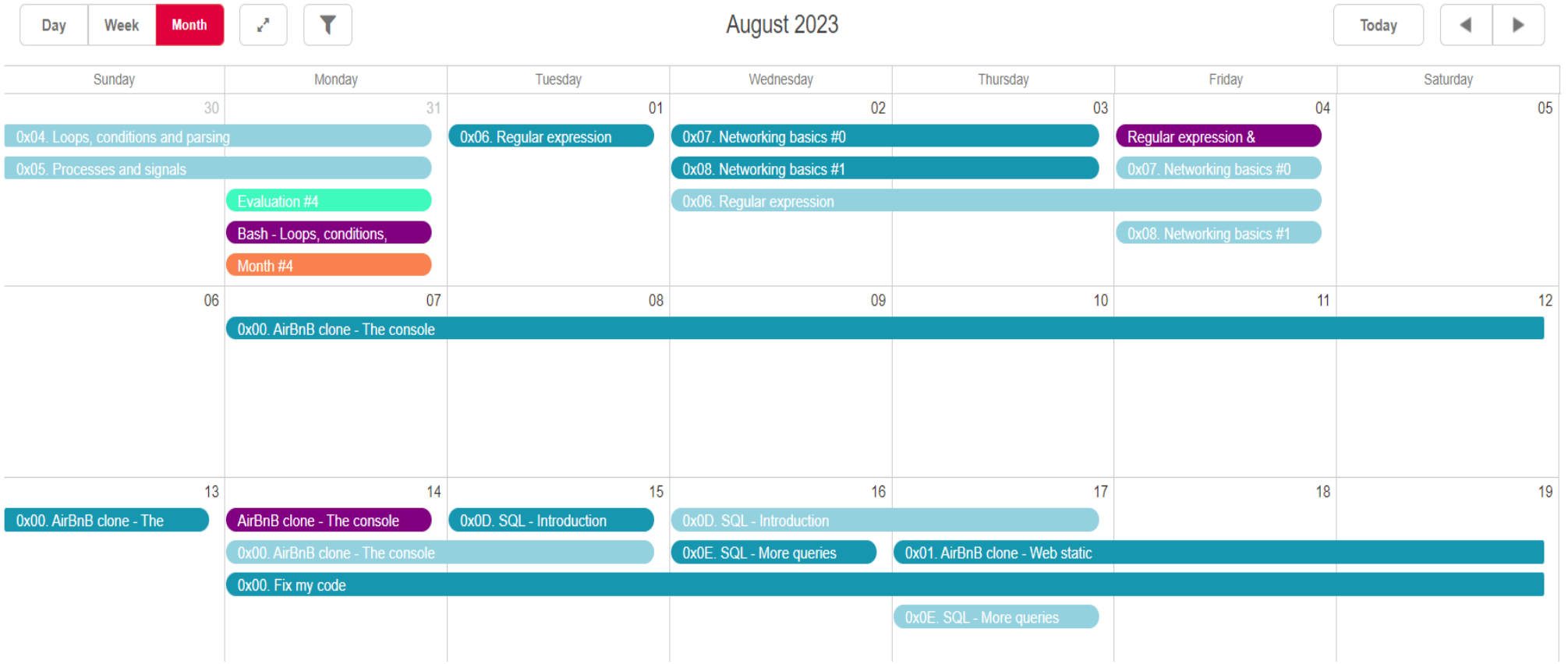

Day
Week
Month
May 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
30
01
02
03
04
05
06
Professional Social Presence
Workers' Day
C - More singly linked lists
0x14. C - Bit manipulation
C - Bit manipulation
0x13. C - More singly linked lists
0x13. C - More singly linked lists
0x14. C - Bit manipulation
Month #1
07
08
09
10
11
12
13
0x15. C - File I/O
C - File I/O
0x16. C - Simple Shell
0x15. C - File I/O
14
15
16
17
18
19
20
0x16. C - Simple Shell
21
22
23
24
25
26
27
0x16. C - Simple Shell
C - Simple Shell - Manual
Evaluation #2
Break #0
0x16. C - Simple Shell
C - Simple Shell
Month #2
28
29
30
31
01
02
Break #0

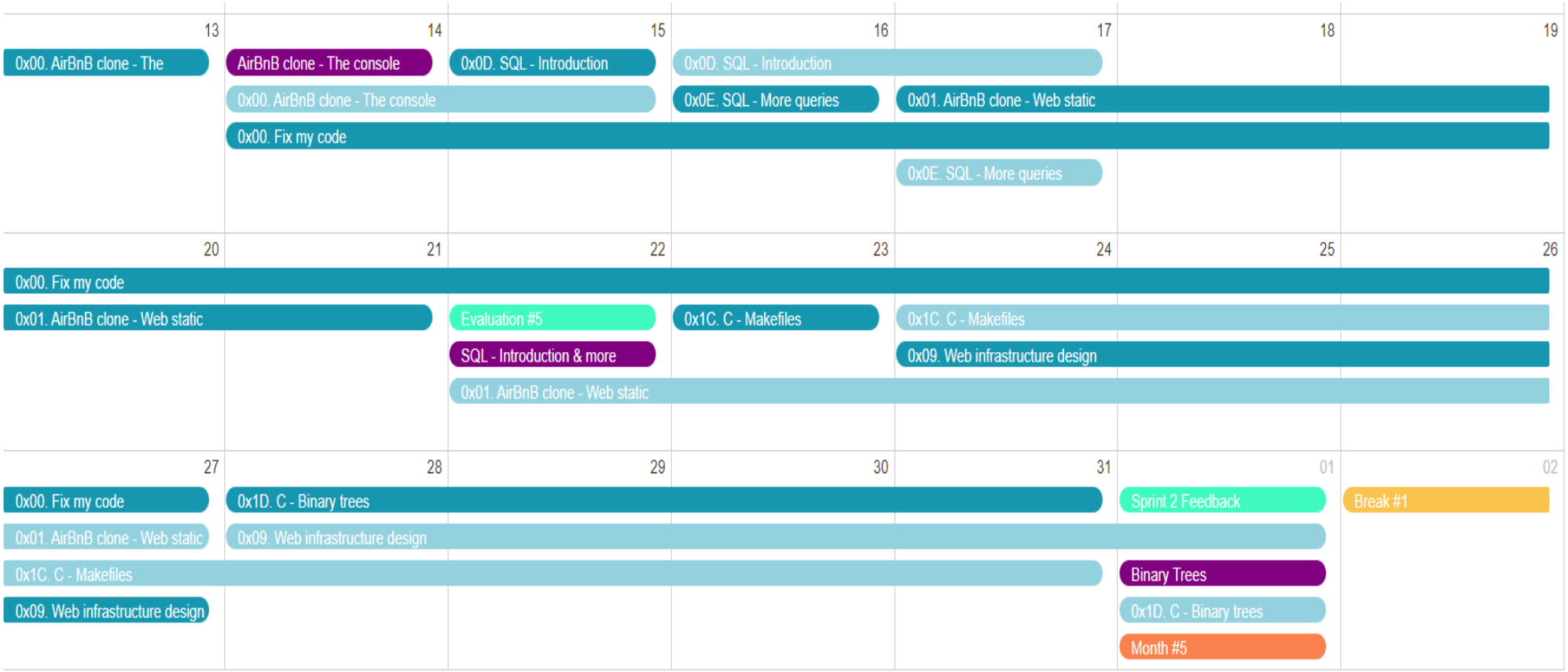

04
Break #0
05
0x00. Python - Hello, World
06
0x00. Python - Hello, World
0x01. Python - if/else, loops,
07
Python - Hello, World & if/else,
0x01. Python - if/else, loops,
08
0x02. Python - import &
09
0x02. Python - import & modules
0x03. Python - Data Structures: Lists, Tuples
10
11
0x02. Python - import & modules
0x03. Python - Data Structures: Lists, Tuples
12
13
C - Python - import & modules
0x03. Python - Data Structures:
14
0x04. Python - More Data
15
0x17. C - Doubly linked lists
0x04. Python - More Data Structures: Set, Dictionary
16
C - Doubly linked lists & Python
0x17. C - Doubly linked lists
17
18
19
0x18. C - Dynamic libraries
RSA Factoring Challenge
20
0x18. C - Dynamic libraries
0x19. C - Stacks, Queues - LIFO, FIFO
21
22
23
C - Stacks, Queues - LIFO,
0x19. C - Stacks, Queues - LIFO, FIFO
24
25
RSA Factoring Challenge
0x19. C - Stacks, Queues -
26
0x05. Python - Exceptions
27
0x05. Python - Exceptions
0x06. Python - Classes and
28
Evaluation #3
Python - Exceptions & Classes
0x07. Python - Test-driven development
0x06. Python - Classes and
Month #3
29
Id el Kabir
30
01

My planning

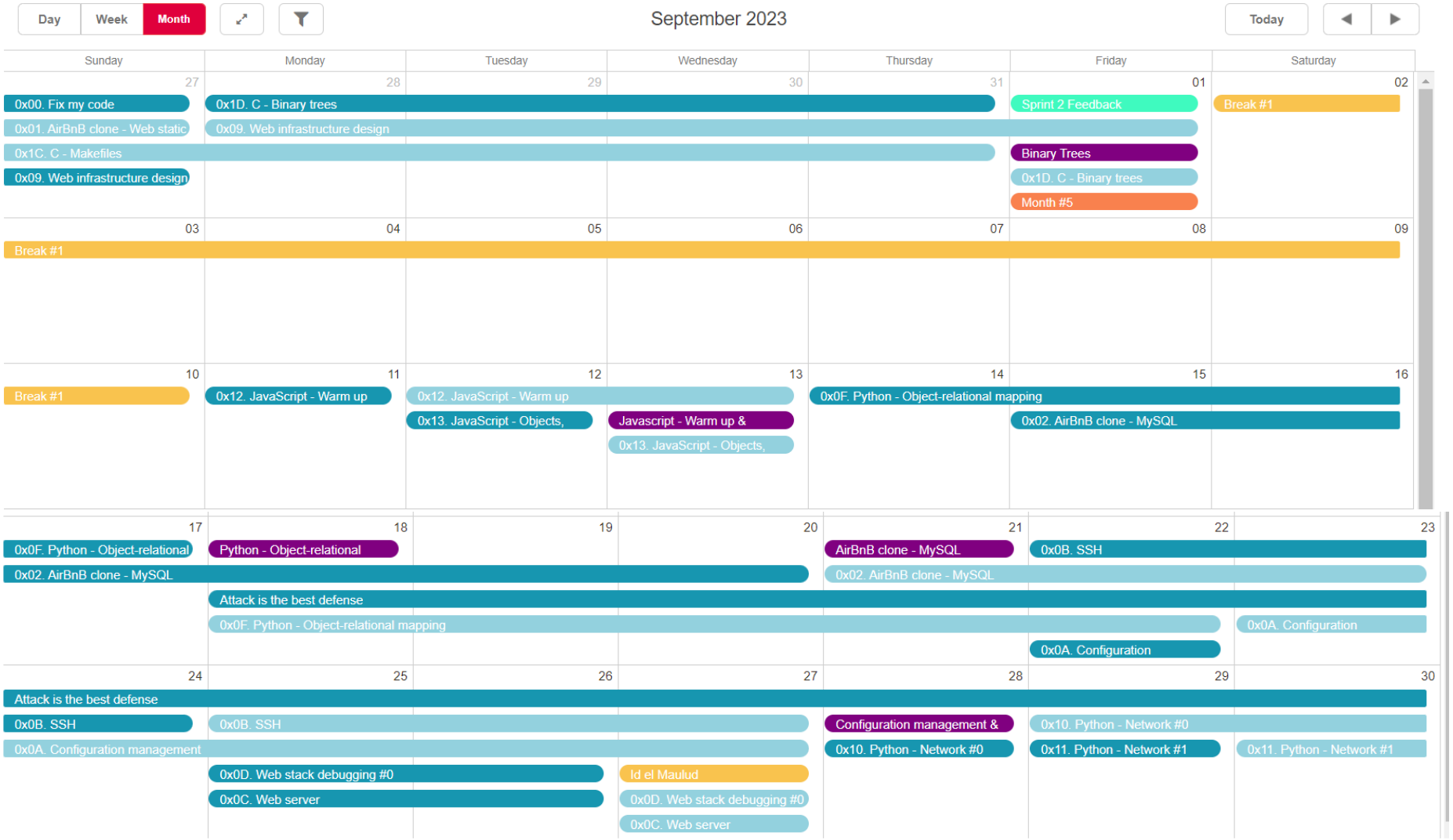

Day
Week
Month
July 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
25
26
27
28
29
30
01
RSA Factoring Challenge
0x19. C - Stacks, Queues -
0x05. Python - Exceptions
0x05. Python - Exceptions
0x06. Python - Classes and
Id el Kabir
Evaluation #3
Python - Exceptions & Classes
0x07. Python - Test-driven development
0x06. Python - Classes and
Month #3
02
03
04
05
06
07
08
RSA Factoring Challenge
0x08. Python - More Classes
0x08. Python - More Classes and Objects
0x1A. C - Hash tables
0x1A. C - Hash tables
0x07. Python - Test-driven development
Python - Test-driven
0x09. Python - Everything is
0x07. Python - Test-driven development
0x09. Python - Everything is
09
10
11
12
13
14
15
0x1A. C - Hash tables
0x0C. Python - Almost a circle
0x0A. Python - Inheritance
0x0A. Python - Inheritance
Command line for the win
0x0B. Python - Input/Output
C - Hash tables & Python -
0x0B. Python - Input/Output
16
17
18
19
20
21
22
Command line for the win
0x0C. Python - Almost a circle
Python - Almost a circle
0x1B. C - Sorting algorithms & Big O
0x0C. Python - Almost a circle
23
24
25
26
27
28
29
Command line for the win
0x1B. C - Sorting algorithms & Big O
C - Sorting algorithms & Big O
0x04. Loops, conditions and
0x04. Loops, conditions and parsing
0x1B. C - Sorting algorithms & Big O
0x05. Processes and signals
0x05. Processes and signals
30
31
01
02
03
04
05
0x04. Loops, conditions and parsing
0x05. Processes and signals
0x06. Regular expression
0x07. Networking basics #0
0x08. Networking basics #1
Regular expression &
0x07. Networking basics #0
Evaluation #4
Bash - Loops, conditions,
Month #4
0x06. Regular expression
0x08. Networking basics #1

My planning

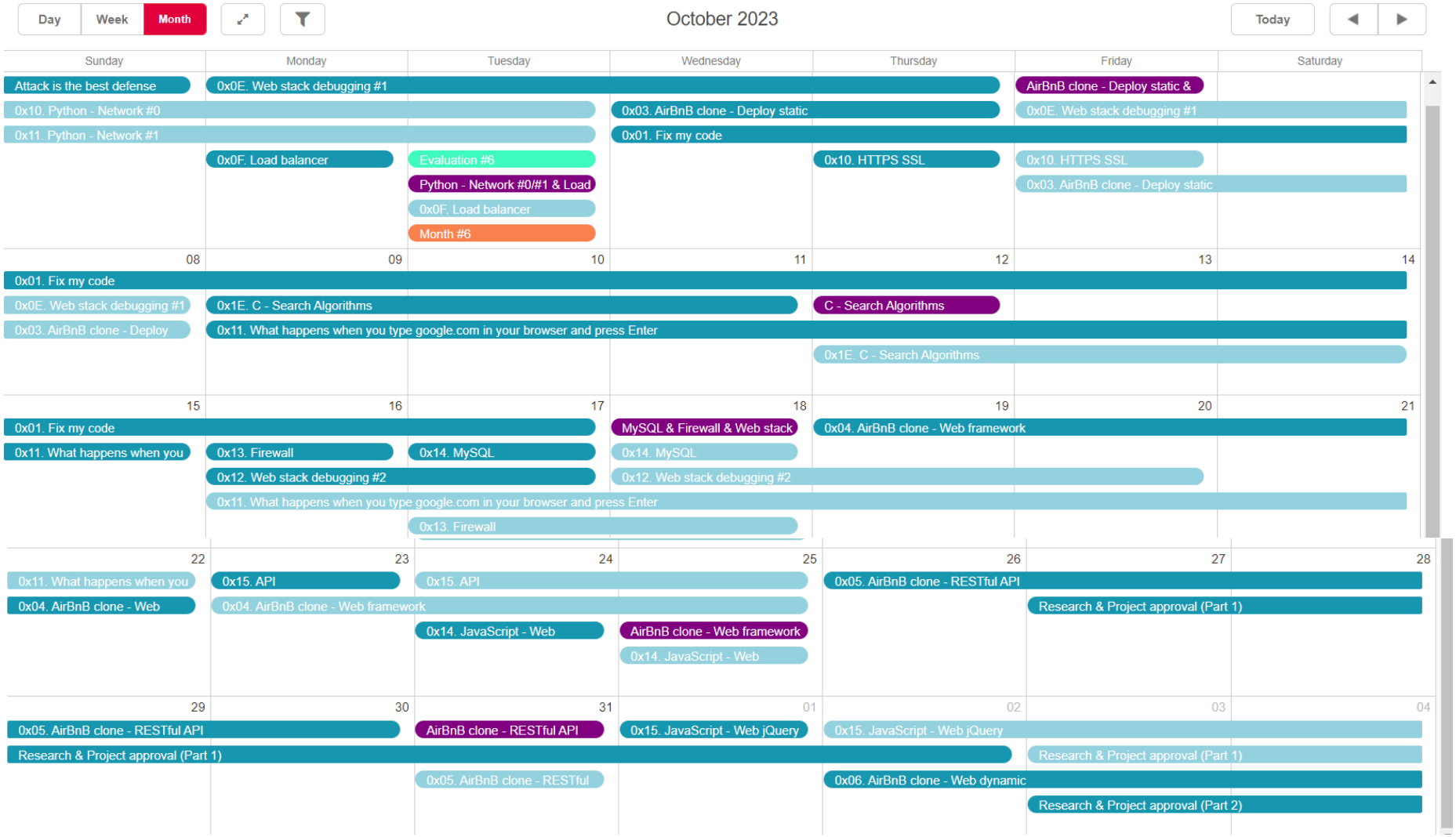

Day
Week
Month
August 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
30
31
01
02
03
04
05
0x04. Loops, conditions and parsing
0x05. Processes and signals
Evaluation #4
Bash - Loops, conditions,
Month #4
0x06. Regular expression
0x07. Networking basics #0
0x08. Networking basics #1
0x06. Regular expression
Regular expression &
0x07. Networking basics #0
0x08. Networking basics #1
06
07
08
09
10
11
12
0x00. AirBnB clone - The console
13
14
15
16
17
18
19
0x00. AirBnB clone - The
AirBnB clone - The console
0x0D. SQL - Introduction
0x0D. SQL - Introduction
0x00. AirBnB clone - The console
0x0E. SQL - More queries
0x01. AirBnB clone - Web static
0x00. Fix my code
0x0E. SQL - More queries


13
0x00. AirBnB clone - The
14
AirBnB clone - The console
0x00. AirBnB clone - The console
0x00. Fix my code
15
0x0D. SQL - Introduction
16
0x0D. SQL - Introduction
0x0E. SQL - More queries
17
0x01. AirBnB clone - Web static
0x0E. SQL - More queries
18
19
20
0x00. Fix my code
0x01. AirBnB clone - Web static
21
22
Evaluation #5
SQL - Introduction & more
0x01. AirBnB clone - Web static
23
0x1C. C - Makefiles
24
0x1C. C - Makefiles
0x09. Web infrastructure design
25
26
27
0x00. Fix my code
0x01. AirBnB clone - Web static
0x1C. C - Makefiles
0x09. Web infrastructure design
28
0x1D. C - Binary trees
0x09. Web infrastructure design
29
30
31
01
Sprint 2 Feedback
Binary Trees
0x1D. C - Binary trees
Month #5
02
Break #1

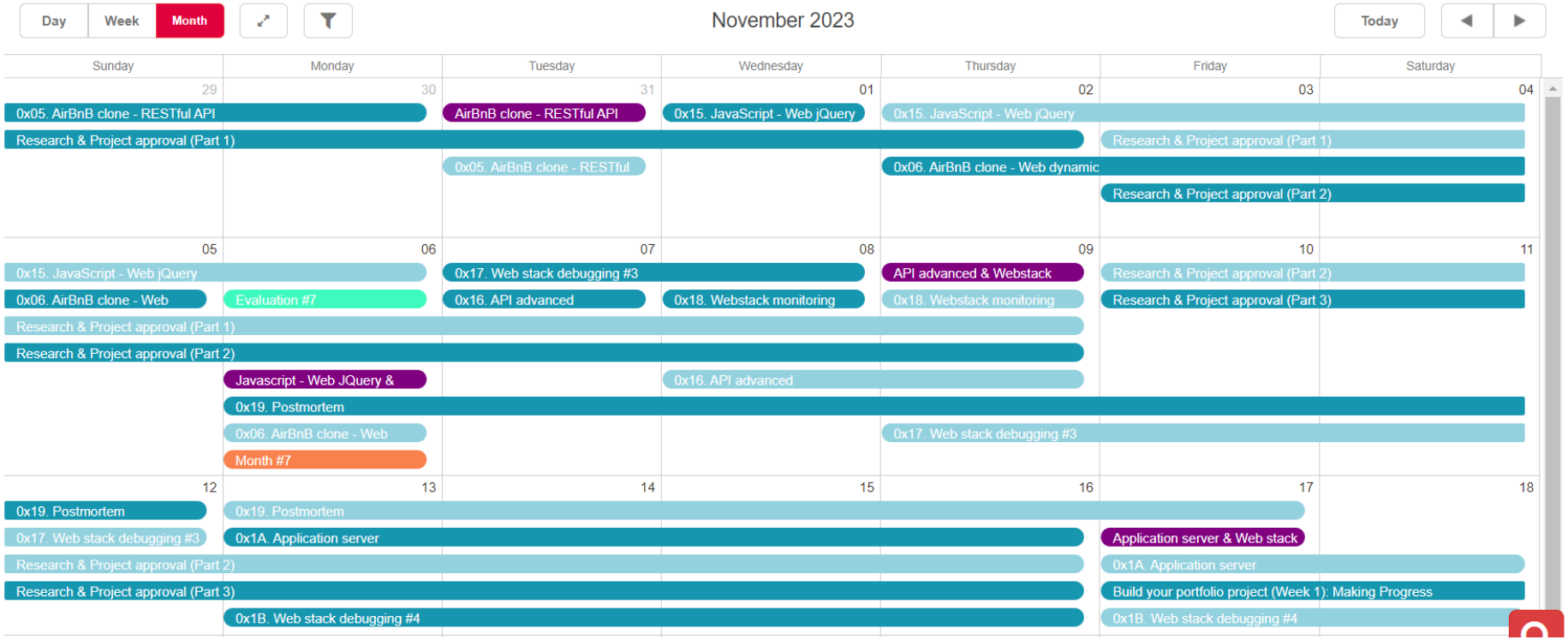
My planning

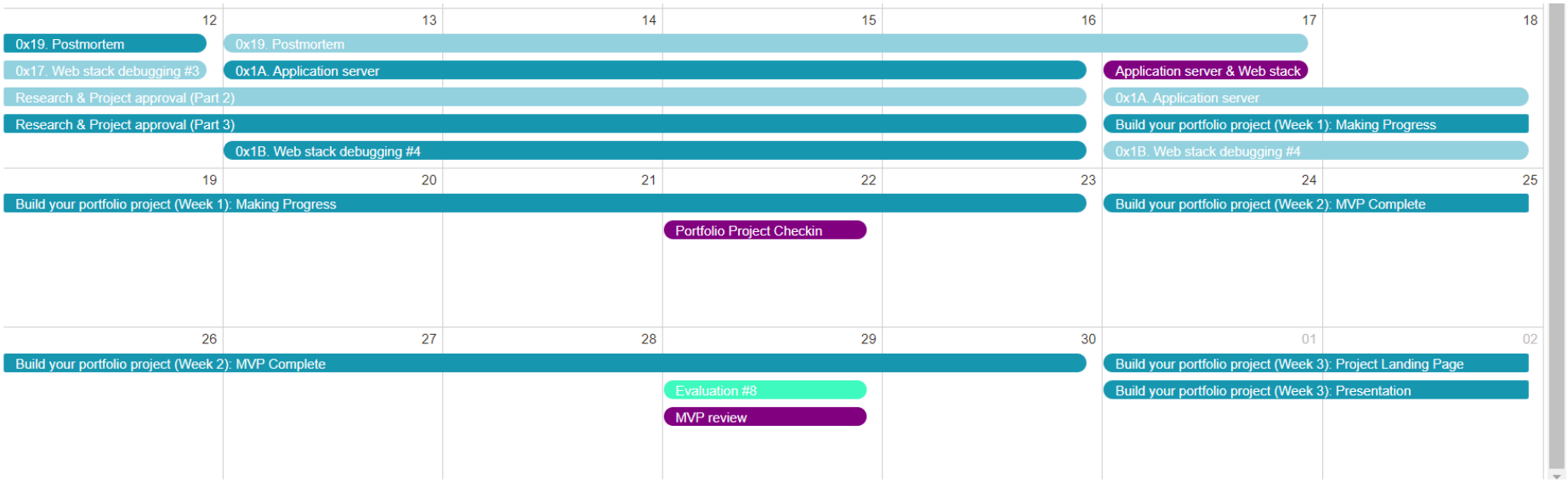

Day
Week
Month
September 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
27
0x00. Fix my code
0x01. AirBnB clone - Web static
0x1C. C - Makefiles
0x09. Web infrastructure design
28
0x1D. C - Binary trees
0x09. Web infrastructure design
29
30
31
01
Sprint 2 Feedback
Binary Trees
0x1D. C - Binary trees
Month #5
02
Break #1
03
Break #1
04
05
06
07
08
09
10
Break #1
11
0x12. JavaScript - Warm up
12
0x12. JavaScript - Warm up
0x13. JavaScript - Objects,
13
Javascript - Warm up &
0x13. JavaScript - Objects,
14
0x0F. Python - Object-relational mapping
15
0x02. AirBnB clone - MySQL
16
17
0x0F. Python - Object-relational
0x02. AirBnB clone - MySQL
18
Python - Object-relational
Attack is the best defense
0x0F. Python - Object-relational mapping
19
20
21
AirBnB clone - MySQL
0x02. AirBnB clone - MySQL
22
0x0B. SSH
0x0A. Configuration
23
0x0A. Configuration
24
Attack is the best defense
0x0B. SSH
0x0A. Configuration management
25
0x0B. SSH
0x0D. Web stack debugging #0
0x0C. Web server
26
27
Id el Maulud
0x0D. Web stack debugging #0
0x0C. Web server
28
Configuration management &
0x10. Python - Network #0
29
0x10. Python - Network #0
0x11. Python - Network #1
30
0x11. Python - Network #1

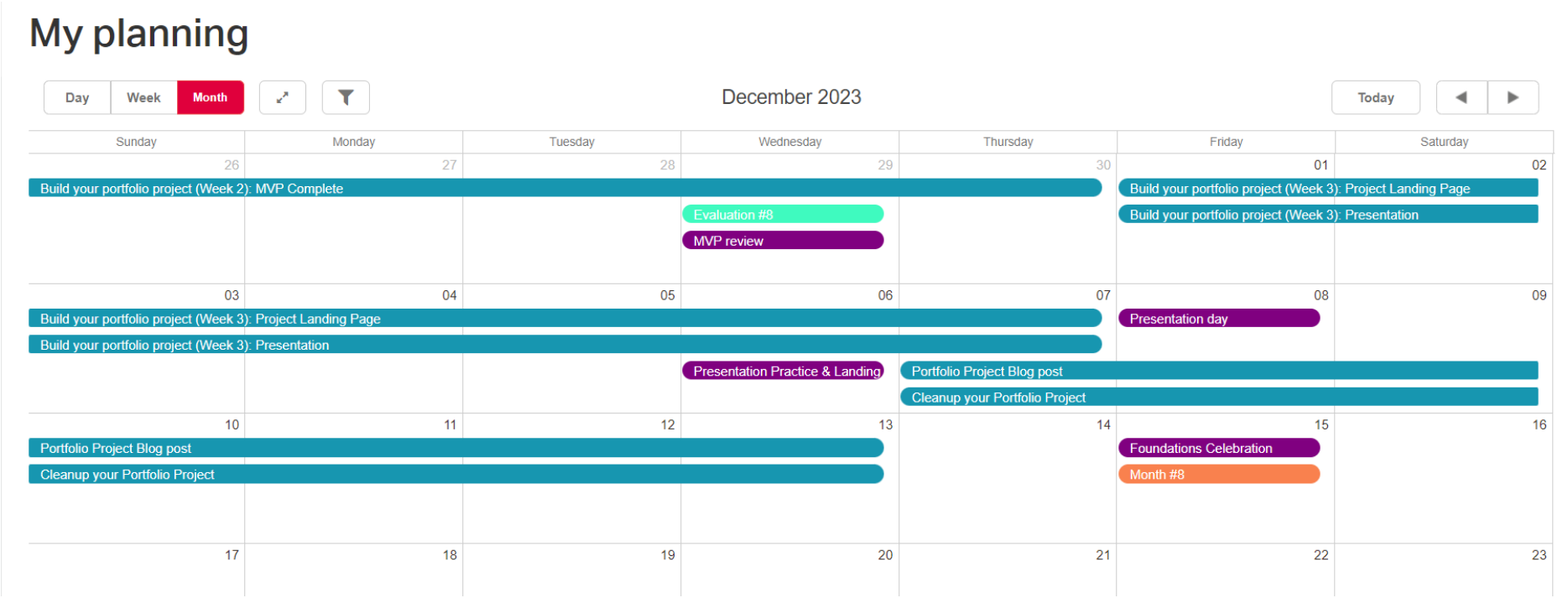
My planning


Day
Week
Month
October 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
Attack is the best defense
0x0E. Web stack debugging #1
AirBnB clone - Deploy static &
0x10. Python - Network #0
0x03. AirBnB clone - Deploy static
0x0E. Web stack debugging #1
0x11. Python - Network #1
0x01. Fix my code
0x0F. Load balancer
Evaluation #6
0x10. HTTPS SSL
0x10. HTTPS SSL
Python - Network #0/#1 & Load
0x03. AirBnB clone - Deploy static
0x0F. Load balancer
Month #6
08
09
10
11
12
13
14
0x01. Fix my code
0x0E. Web stack debugging #1
0x1E. C - Search Algorithms
C - Search Algorithms
0x03. AirBnB clone - Deploy
0x11. What happens when you type google.com in your browser and press Enter
0x1E. C - Search Algorithms
15
16
17
18
19
20
21
0x01. Fix my code
MySQL & Firewall & Web stack
0x04. AirBnB clone - Web framework
0x11. What happens when you
0x13. Firewall
0x14. MySQL
0x14. MySQL
0x12. Web stack debugging #2
0x12. Web stack debugging #2
0x11. What happens when you type google.com in your browser and press Enter
0x13. Firewall
22
23
24
25
26
27
28
0x11. What happens when you
0x15. API
0x15. API
0x05. AirBnB clone - RESTful API
0x04. AirBnB clone - Web
0x04. AirBnB clone - Web framework
Research & Project approval (Part 1)
0x14. JavaScript - Web
AirBnB clone - Web framework
0x14. JavaScript - Web
29
30
31
01
02
03
04
0x05. AirBnB clone - RESTful API
AirBnB clone - RESTful API
0x15. JavaScript - Web jQuery
0x15. JavaScript - Web jQuery
Research & Project approval (Part 1)
Research & Project approval (Part 1)
0x05. AirBnB clone - RESTful
0x06. AirBnB clone - Web dynamic
Research & Project approval (Part 2)

My planning


Day
Week
Month
November 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
29
30
31
01
02
03
04
0x05. AirBnB clone - RESTful API
AirBnB clone - RESTful API
0x15. JavaScript - Web jQuery
0x15. JavaScript - Web jQuery
Research & Project approval (Part 1)
Research & Project approval (Part 1)
0x05. AirBnB clone - RESTful
0x06. AirBnB clone - Web dynamic
Research & Project approval (Part 2)
05
06
07
08
09
10
11
0x15. JavaScript - Web jQuery
0x17. Web stack debugging #3
API advanced & Webstack
Research & Project approval (Part 2)
0x06. AirBnB clone - Web
Evaluation #7
0x16. API advanced
0x18. Webstack monitoring
0x18. Webstack monitoring
Research & Project approval (Part 3)
Research & Project approval (Part 1)
Research & Project approval (Part 2)
Javascript - Web JQuery &
0x16. API advanced
0x19. Postmortem
0x06. AirBnB clone - Web
0x17. Web stack debugging #3
Month #7
12
13
14
15
16
17
18
0x19. Postmortem
0x19. Postmortem
0x17. Web stack debugging #3
0x1A. Application server
Application server & Web stack
Research & Project approval (Part 2)
0x1A. Application server
Research & Project approval (Part 3)
Build your portfolio project (Week 1): Making Progress
0x1B. Web stack debugging #4
0x1B. Web stack debugging #4


12
13
14
15
16
17
18
0x19. Postmortem
0x19. Postmortem
0x17. Web stack debugging #3
0x1A. Application server
Application server & Web stack
Research & Project approval (Part 2)
0x1A. Application server
Research & Project approval (Part 3)
Build your portfolio project (Week 1): Making Progress
0x1B. Web stack debugging #4
0x1B. Web stack debugging #4
19
20
21
22
23
24
25
Build your portfolio project (Week 1): Making Progress
Build your portfolio project (Week 2): MVP Complete
Portfolio Project Checkin
26
27
28
29
30
01
02
Build your portfolio project (Week 2): MVP Complete
Build your portfolio project (Week 3): Project Landing Page
Evaluation #8
Build your portfolio project (Week 3): Presentation
MVP review


My planning
Day
Week
Month
December 2023
Today
Sunday
Monday
Tuesday
Wednesday
Thursday
Friday
Saturday
26
27
28
29
30
01
02
Build your portfolio project (Week 2): MVP Complete
Build your portfolio project (Week 3): Project Landing Page
Evaluation #8
Build your portfolio project (Week 3): Presentation
MVP review
03
04
05
06
07
08
09
Build your portfolio project (Week 3): Project Landing Page
Presentation day
Build your portfolio project (Week 3): Presentation
Presentation Practice & Landing
Portfolio Project Blog post
Cleanup your Portfolio Project
10
11
12
13
14
15
16
Portfolio Project Blog post
Foundations Celebration
Cleanup your Portfolio Project
Month #8
17
18
19
20
21
22
23